

Sanjay Khati Chhetri

**COMPARATIVE STUDY OF FRONT-END FRAMEWORKS:
REACT and ANGULAR**

Comparative Study of Front-end Frameworks: React, and Angular

Sanjay Khati Chhetri
Bachelor's Thesis
Spring 2024
Information Technology
Oulu University of Applied Sciences

ABSTRACT

Oulu University of Applied Sciences
Information Technology, Web Development

Author(s): Sanjay Khati Chhetri

Title of the thesis: Comparative Study of Front-end Frameworks: React and Angular

Thesis examiner(s): Ville Majava

Term and year of thesis completion: Spring 2024

Pages: 76 + 14 appendices

This thesis presents a comprehensive comparative analysis of two leading front-end frameworks: React and Angular. The study aims to provide developers, project managers, and organizations with insights to make informed decisions when choosing between these frameworks for web application development.

The research employed a multi-faceted approach, combining theoretical analysis with practical implementation. A Todo List application was developed using both React and Angular to provide hands-on insights into each framework's capabilities. The comparison criteria included learning curve, performance, ecosystem and tooling, community support and documentation, and popularity.

Performance testing was conducted using Chrome DevTools, providing objective metrics on initial load times, runtime efficiency, and resource utilization. The study also analysed developer surveys, usage statistics, and trend data to assess the frameworks' popularity and adoption rates.

Key findings reveal that React offers a gentler learning curve and superior performance in terms of initial load times and memory usage. Its flexibility and extensive ecosystem make it suitable for a wide range of projects, particularly those requiring rapid development. Angular, with its comprehensive feature set and structured approach, demonstrated strengths in handling large datasets and providing a standardized development environment, making it well-suited for complex, enterprise-level applications.

The research also highlighted the robust community support and documentation available for both frameworks, with React boasting a larger and more diverse community, while Angular offers more focused, enterprise-oriented resources.

This study contributes to the ongoing discourse on front-end development technologies by providing a thorough, objective comparison of React and Angular. The findings can guide technology selection based on project requirements, team expertise, and organizational goals in the rapidly evolving landscape of web development.

Keywords: React, Angular, Front-end Frameworks, Web Development, Performance Analysis, Developer Experience, Ecosystem Comparison.

CONTENTS

1	INTRODUCTION	8
1.1	Objective	8
2	FRAMEWORK OVERVIEW	10
2.1	React.....	13
2.1.1	Core Concepts of React	13
2.1.2	Architecture.....	17
2.1.3	Best suited use case.....	18
2.2	Angular.....	19
2.2.1	Core Concepts of Angular	19
2.2.2	Architecture.....	24
2.2.3	Best suited use case.....	25
3	CRITERIA FOR RESEARCH	27
3.1	Comparison criteria	27
3.2	Methodology.....	28
3.2.1	Justification for Methodology	28
4	IMPLEMENTATION.....	29
4.1	Application Development	29
4.2	Backend Architecture	29
4.2.1	Core Functionalities	29
4.3	React Implementation	30
4.3.1	Architecture of React TodoApp:.....	30
4.3.2	Key Features	31
4.4	Angular Implementation	33
4.4.1	Architecture of Angular TodoApp.....	33
4.4.2	Key Feature	34
4.5	Comparative Observations.....	36
4.5.1	Development Environment and Setup	36
4.5.2	Component Architecture	37
4.5.3	State Management	37
4.5.4	TypeScript Integration.....	37
4.5.5	API Integration	38

4.5.6	Routing	38
4.5.7	Form Handling	38
5	ANALYSIS	40
5.1	Learning Curve.....	40
5.1.1	Initial Setup Experience	41
5.1.2	Complexity of Core Concepts	41
5.1.3	Time to Productivity	42
5.1.4	Availability and Quality of Learning Resources.....	42
5.1.5	Comparative Analysis	43
5.2	Performance Analysis	43
5.2.1	Initial Load Times and Runtime Performance	45
5.2.2	Operation-Specific Performance.....	46
5.2.3	Bundle Size.....	48
5.2.4	Code Efficiency.....	49
5.2.5	Memory Usage	51
5.2.6	Conclusion:.....	53
5.3	Ecosystem and Tooling.....	55
5.3.1	React Ecosystem and Tooling:	55
5.3.2	Angular Ecosystem and Tooling:	56
5.3.3	Comparative Analysis:	56
5.4	Community Support and Documentation Analysis	57
5.4.1	React Community Support and Documentation:.....	57
5.4.2	Angular Community Support and Documentation:.....	58
5.4.3	Comparative Analysis:	59
5.5	Popularity Analysis.....	60
5.5.1	Developer Surveys and Usage Statistics:.....	60
5.5.2	Google Trends Analysis:.....	62
5.5.3	Website Usage Statistics:	62
5.5.4	Comparative Analysis:	63
5.5.5	Conclusion:	63
5.6	Limitations of the scope of Analysis	64
6	DISCUSSION	66
6.1	Learning Curve and Developer Experience	66
6.2	Performance and Scalability	67

6.3	Ecosystem and Tooling.....	67
6.4	Community Support and Documentation	68
6.5	Popularity and Industry Adoption	68
6.6	Limitations and Future Research	69
6.7	Conclusion	70
REFERENCES		71
APPENDICES.....		75

ABBREVIATIONS

AJAX: Asynchronous JavaScript and XML

API: Application Programming Interface

CLI: Command Line Interface

CLS: Cumulative Layout Shift

CPU: Central Processing Unit

CRUD: Create, Read, Update, Delete

CSS: Cascading Style Sheets

DI: Dependency Injection

DOM: Document Object Model

FCP: First Contentful Paint

FPS: Frames Per Second

HTML: Hypertext Markup Language

HTTP: Hypertext Transfer Protocol

HTTPS: Hypertext Transfer Protocol Secure

JS: JavaScript

JSX: JavaScript XML

LCP: Largest Contentful Paint

MVC: Model-View-Controller

NPM: Node Package Manager

ORM: Object-Relational Mapping

PWAs: Progressive Web Applications

RxJS: Reactive Extensions for JavaScript

SDLC: Software Development Lifecycle

SOLID: Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion

SPAs: Single Page Applications

TBT: Total Blocking Time

TTI: Time to Interactive

UI: User Interface

UIs: User Interfaces

1 INTRODUCTION

In the ever-changing landscape of web development, the emergence of front-end frameworks has marked a significant evolution in how web applications are developed, maintained, and enhanced. Among the plethora of options available, React and Angular stand out due to their robust features, scalability, and widespread adoption by the developer community. This thesis conducts a comparative study of these two leading frameworks, aiming to delineate their strengths, weaknesses, and optimal use cases from a practical perspective.

React and Angular, developed by Facebook and Google respectively, have carved their niches within the development community. React, with its declarative component-based architecture approach and efficient virtual DOM implementation, offers a lightweight solution for building user interfaces. Angular, on the other hand, presents a full-fledged framework that embraces TypeScript, providing an extensive suite of tools for developing large scale applications. The motivation behind selecting these two frameworks for comparison stems from their wide adoption and the diverse discussions within the developer community regarding their applicability, performance, and learning curve.

The significance of this comparative study extends beyond academic curiosity. It serves as a pivotal resource for developers, project managers, and organizations, facilitating informed decision-making when selecting a front-end framework for their web development needs. Additionally, this research aims to contribute to the broader discourse on web development technologies, providing fresh insights and practical evaluations based on empirical evidence.

1.1 Objective

The primary objective of this research is to provide a comprehensive comparison between React and Angular, two of the leading front-end frameworks in the field of web development. This comparison aims to evaluate the frameworks across several dimensions—learning curve, performance, ecosystem and tooling, community support, and documentation. Through the lens of a practical

implementation, this thesis constructs a Todo List application utilizing both frameworks to offer a grounded perspective on their use.

This comparative analysis seeks to shed light on the nuanced differences and potential use cases for each framework, thereby assisting developers in making informed choices that align with the specific requirements of their projects. By doing so, this study endeavours to contribute valuable insights to the ongoing discussion within the web development community regarding the optimal application of these technologies in real-world scenarios.

2 FRAMEWORK OVERVIEW

The evolution of web development has been significantly influenced by the continuous pursuit of more dynamic, responsive, and intuitive user interfaces. This quest has led to the emergence of front-end frameworks, a cornerstone in the modern development landscape. The Framework Overview section is designed to shed light on the integral role these technologies play, their historical evolution, and their impact on the development of web applications today and in the future.

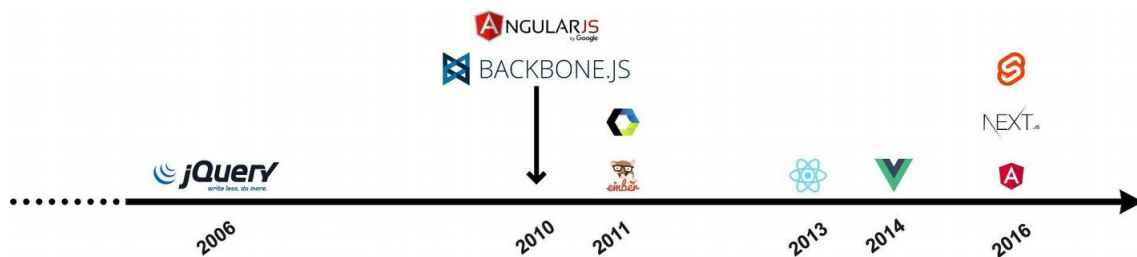


Figure 1. Evolution of JavaScript frontend frameworks (Strazzullo, 2023).

The need for and importance of front-end frameworks lies in their ability to abstract the complexity of directly manipulating the DOM (Document Object Model), managing state changes, and ensuring responsive and accessible UIs. By standardizing these underlying mechanisms, frameworks free developers to focus more on creating unique features and user experiences rather than reinventing the wheel for common functionalities.

The landscape of web development has undergone monumental shifts over the past few decades, evolving from static HTML pages to complex, dynamic web applications that rival native apps in functionality and user experience. This journey has been marked by the introduction of various front-end frameworks, each response to the challenges and needs of its time.

1996-2000: The Early Days - Web design in the very early 1990s was all about sharing information and websites were often pure text, with blue hyperlinks that turned purple on click(2). The introduction of CSS in 1996(3) and JavaScript(4) allowed for more stylistic and interactive elements but was still limited in scope. The need for more dynamic content led to the use of technologies like Flash and Java Applets, which provided interactivity but lacked accessibility and standards compliance.

2006: jQuery - In 2006, as websites were becoming increasingly interactive, the constraints of raw JavaScript and inconsistencies across different browsers started to become evident. This is where jQuery, a JavaScript library developed by John Resig, stepped in (5). The true value of jQuery lay in its ability to establish a common language, or a 'lingua franca', among different browsers. This facilitated the growth of a community around a shared understanding, apart from the selector syntax. During this period, numerous features were integrated into the main project. This encompassed AJAX requests, animations, and other utilities, which significantly reduced the complexity of scripts (6). Moreover, jQuery played a pivotal role in promoting the wider adoption of dynamic content (7). By simplifying HTML document traversal, event handling, and animation, it made it easier to manipulate HTML documents, handle events, perform animations, and much more. As a result, jQuery has had a profound impact on the front-end world, shaping the way we design and interact with websites today.

2010: AngularJS - In 2010, Google launched AngularJS, acknowledging the constraints of jQuery for constructing large-scale dynamic applications (8). This marked the advent of Single Page Applications (SPAs), a new era in web development (9). AngularJS offered a robust and organized framework, featuring two-way data binding, MVC architecture, and dependency injection. These features streamlined the development process, promoting organization and efficiency. As a result, developers were able to create highly interactive web applications. This shift in web development practices significantly impacted how websites were designed and interacted with, paving the way for more dynamic and user-friendly web experiences.

2013: React – In 2013, Facebook introduced React, a transformative approach to User Interface (UI) rendering. The key innovation was the introduction of the virtual DOM (10), an optimization that significantly improved the performance of highly dynamic and data-intensive applications. React's component-based architecture further empowered developers, enabling them to construct complex UIs using reusable, stateful components (11). This approach fostered enhanced maintainability and scalability, marking a significant advancement in the field.

2014: Vue.js - In 2014, Vue.js was introduced, incorporating the insights gained from its predecessors (12). It presented a progressive framework that was both lightweight and versatile. Vue.js emphasized the view layer, making it particularly suitable for enhancing user interface (UI) development (13). One of the key features of Vue.js was its ease of integration. This made it possible to

incorporate Vue.js into ongoing projects without significant overhaul. Furthermore, Vue.js was designed with an incrementally adoptable architecture, allowing developers to adopt its features step by step based on their needs (14). Vue.js effectively addressed the complexities associated with developing Single Page Applications (SPAs). Despite this, it maintained enough flexibility to be integrated with other libraries or existing projects

2016: Angular2 – In 2016, Angular2 marked a significant pivot in the framework's direction (15). Released by Google, this complete rewrite introduced a component-based architecture similar to React but leveraged TypeScript for added type safety and better tooling (16). Angular's robustness made it an attractive solution for enterprise-level applications, offering a comprehensive set of features for complex application ecosystems.

The story of front-end frameworks is one of evolution, driven by the unrelenting pursuit of more dynamic, responsive, and user-centric web applications. This journey witnesses' humble beginnings with tools like jQuery simplifying DOM manipulation, progressing through the establishment of structured, comprehensive frameworks such as AngularJS, which introduced concepts like two-way data binding and dependency injection. The advent of React further revolutionized the landscape with its virtual DOM, paving the way for the creation of highly performant applications capable of managing dynamic data efficiently. These frameworks not only altered the web development process by emphasizing modularity, scalability, and maintainability but also cultivated robust communities that have been instrumental in fostering innovation and best practices.

Looking towards the future, front-end frameworks are certain to continue their evolution, embracing web standards like Web Components for enhanced modularity, and prioritizing performance through techniques like server-side rendering. The increasing importance of Progressive Web Apps (PWAs) underscores a push towards applications that deliver seamless, native-like experiences across all platforms. Moreover, the integration of emerging web capabilities, such as Web Assembly, hints at a future where web applications can achieve near-native performance. As these frameworks evolve, they promise to offer more holistic development tools that streamline the development lifecycle. The trajectory of front-end frameworks is a testament to their foundational role in the web's ongoing transformation, ensuring they remain at the forefront of creating innovative, efficient, and engaging web applications.

2.1 React

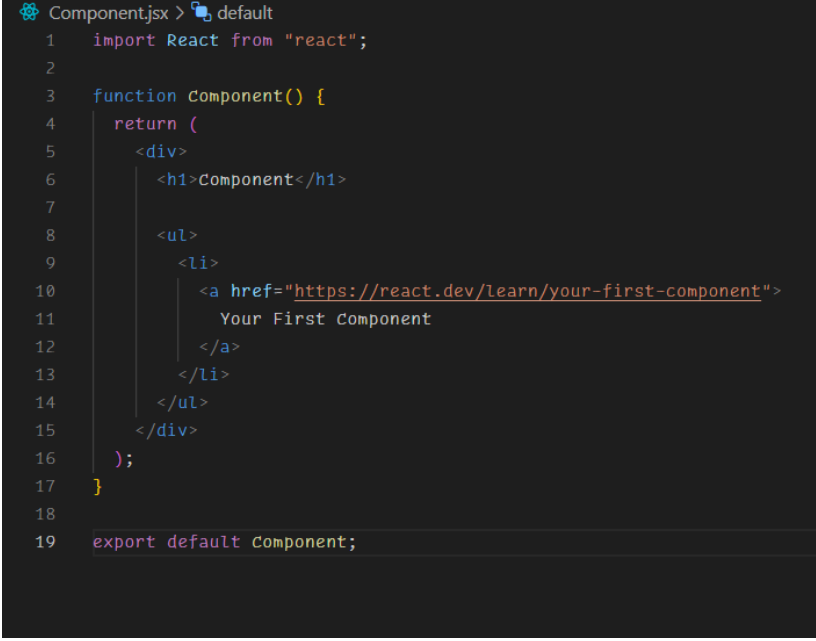
React, created by Facebook software engineer Jordan Walke, represents a major advancement in web development. It was birthed from Facebook's necessity to enhance the user interface for its ever-growing platform, particularly for its News Feed feature. React's origins date back to 2011, when it was initially implemented on Facebook's News Feed. It was later adopted by Instagram in 2012, demonstrating its effectiveness in handling dynamic content and large-scale user interactions. In May 2013, react was officially open sourced at JSConf US, marking a pivotal moment for web developers worldwide(17). This move allowed developers outside Facebook to explore, adapt, and contribute to React, fostering a vibrant and collaborative community. The decision to open-source React was driven by Facebook's vision to democratize UI development, making efficient and scalable front-end development accessible to everyone.

React introduction to the open-source community catalysed its adoption across varied sectors, attributed to its revolutionary approach to building user interfaces. React core principles, such as its virtual DOM and one-way data flow, were game changers, offering a solution to the performance bottlenecks prevalent in existing frameworks and libraries. These technical innovations underpin React ability to efficiently update and render UIs, which, combined with its component-based architecture, differentiated it from its counterparts. The subsequent years saw rapid iterations and enhancements, with significant contributions from both Facebook engineers and the open-source community. The introduction of features such as React Hooks in 2018 at React Conf (18) further exemplified React commitment to simplicity and functionality, allowing developers to leverage state, and lifecycle features within functional components.

2.1.1 Core Concepts of React

These core concepts form the backbone of React development, striking a balance between efficiency and flexibility. Understanding these principles is crucial for developers to effectively leverage React in their projects and in defining React's architecture, whether they are building simple interactive interfaces or complex large-scale applications.

- **JSX (JavaScript XML):** JSX is a syntax extension for JavaScript that resembles HTML. It enables developers to embed HTML structures within JavaScript code, which React subsequently converts into DOM elements. This blending of markup with logic facilitates a more intuitive and cohesive development process, where the UI design closely aligns with the underlying business logic.



```
Component.jsx > default
1  import React from "react";
2
3  function Component() {
4    return (
5      <div>
6        <h1>Component</h1>
7
8        <ul>
9          <li>
10             <a href="https://react.dev/learn/your-first-component">
11               Your First Component
12             </a>
13           </li>
14         </ul>
15       </div>
16     );
17   }
18
19   export default Component;
```

Figure 1. JSX in react

- **Components:** React is all about components. Components are independent and reusable bits of code that define portions of the UI. In React, everything from a simple button to an entire application can be a component. This modular methodology facilitates the management and scaling of complex applications by decomposing them into smaller, manageable components. These components can be developed, tested, and debugged independently.

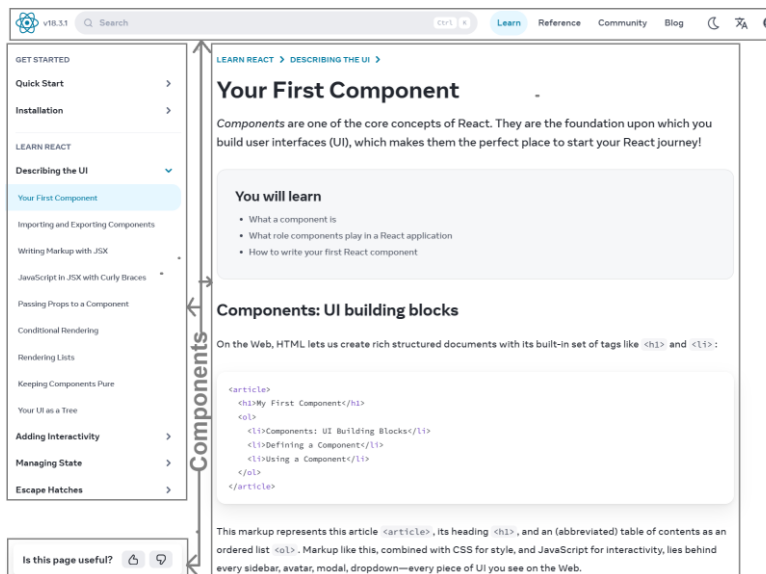


Figure 2. Components

- **Virtual DOM:** React implements a virtual DOM to optimize rendering and improve application performance. Rather than directly manipulating the browser's DOM with each change, React generates a virtual representation of the DOM. When a component's state changes, React updates this virtual DOM first and then computes the differences between the previous and current states of the virtual DOM to update the actual DOM as efficiently as possible.

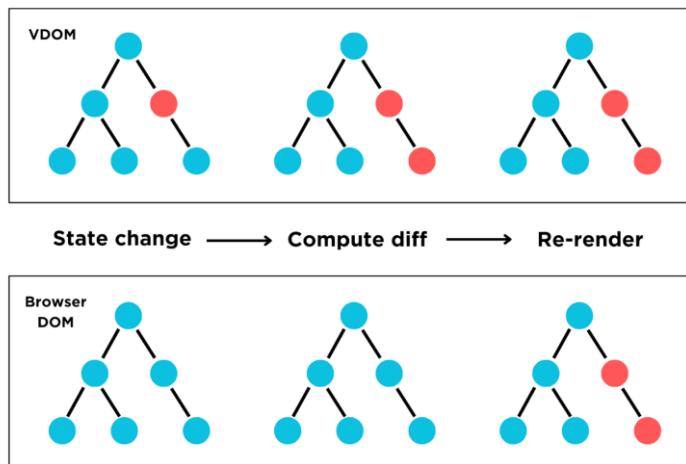


Figure 3. Virtual Dom (19)

- **One-Way Data Binding:** React enforces one-way data flow, meaning data has a single flow direction which makes the application more predictable and easier to understand. State is often passed to child components via props, making parent components control

the data flow. This unidirectional flow ensures a cleaner separation of concerns and better control over the application state.

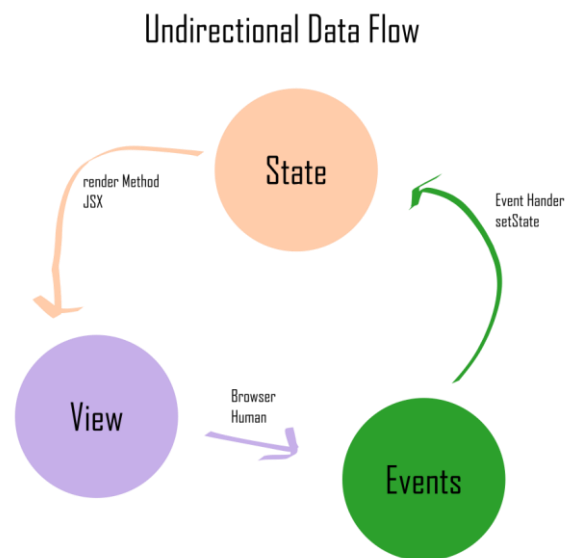


Figure 4. Example of Data Binding (20)

- React Hooks:** Introduced in React 16.8, Hooks are functions that enable developers to access React's state and lifecycle features within function component. Hooks simplify the process of writing components by making state and React features like effects accessible without needing to use classes, thus leading to cleaner and more concise code.

v1.3.1 github.com/donavon/hook-flow

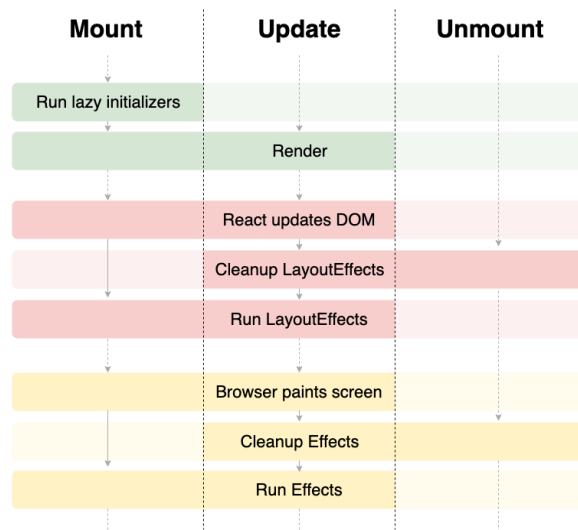


Figure 5. React Hook Flow Diagram (21)

- **State and Lifecycle:** React components have a lifecycle and react provides lifecycle methods that can override to run code at specific times in the process. In addition, components can have internal state, allowing them to maintain data that may change over time and change as per user inputs, network responses, etc., making the application dynamic and interactive.

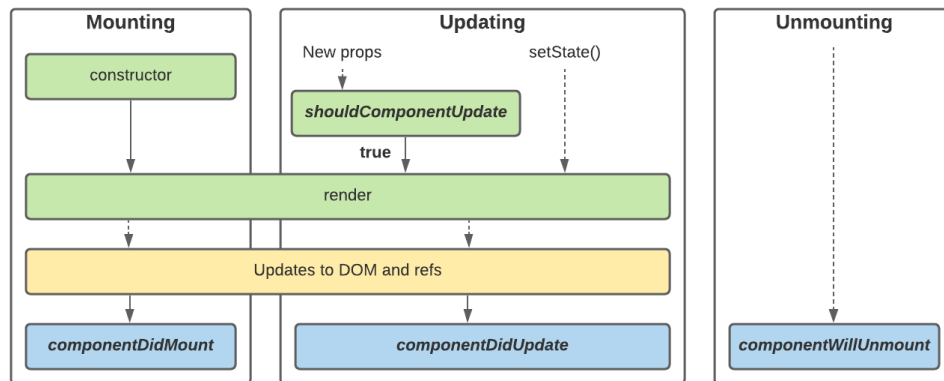


Figure 6. State and Lifecycle in React (22)

2.1.2 Architecture

React's architecture stands out for its unique and efficient approach to building user interfaces. Focused on a component-based architecture, React promotes the creation of user interfaces as a set of independent, reusable components. Each component encapsulates its own state and logic, allowing for the composition of complex UIs with relative ease. A fundamental aspect of React's architecture is the Virtual DOM. Rather than directly manipulating the browser's DOM, which can be inefficient and slow, React employs a virtual representation of the DOM. Consequently, when a component's state changes, React updates this Virtual DOM first. Then, it uses a diffing algorithm to determine what has changed and applies those changes to the actual DOM as efficiently as possible. This process significantly improves application performance, especially in dynamic, data-driven applications. React also emphasizes a unidirectional data flow, wherein data is passed from parent components down to child components through props. This design pattern enhances predictability and debugging by clarifying how data moves through the application. State management further benefits from this model, with state typically living in parent components and being distributed down the hierarchy as needed. With the introduction of hooks in React, state management and lifecycle features for functional components was made way simpler, making them more versatile and allowing for cleaner, more intuitive code structures. This architecture, combined with React's extensive ecosystem, robust community support, and flexibility in integrating with various backend

technologies, makes it a powerful tool for developers creating interactive, efficient, and scalable web applications.

2.1.3 Best suited use case

React is optimally designed to fit a variety of web development needs, thanks to its unique architecture and the fundamental concepts it introduces. Its best-suited use cases encompass a broad spectrum of web applications, from simple interactive UIs to complex, high-performance enterprise solutions. React's adaptability is further evidenced by its widespread adoption across various sectors, such as social media platforms, content management systems, e-commerce websites, and more. Some of the scenarios where React is ideal is such as:

Single-Page Applications (SPAs): React's efficient update mechanism makes it an excellent choice for SPAs, where user experiences akin to a desktop application are required. It can handle complex state management and UI updates without the need for reloading the page, paving the way for fluid, app-like user interactions.

Interactive User Interfaces: Any application that demands rich, interactive UIs can benefit from React. Its declarative component model ensures UI consistency and reliability, making it simpler to develop features like real-time data presentation, dynamic forms, and animations.

High Performance Applications: When application performance is paramount, React's Virtual DOM and smart diffing algorithm allow for minimal and efficient DOM updates, reducing the load time and improving the overall performance of the application.

Large-scale Projects: The modular nature of React, facilitated by its component-based architecture, makes it a solid choice for large projects. It allows for better code organization, reusable components, and easier maintenance and testing.

Complex State Management: For applications that involve complex state management across multiple components and user interactions, React provides a straightforward yet powerful way to manage state at both the component and application level, optionally complemented by state management libraries like Redux or Context API for global state management.

2.2 Angular

Angular, created and supported by Google, is an extensive front-end framework intended for the development of dynamic single-page applications (SPAs). Unlike its predecessor AngularJS, Angular (referred to as Angular 2+) introduced in 2016 (23), transcended as a platform, encompassing a wide array of capabilities aimed at enhancing productivity and facilitating the development of scalable applications.

Angular's inception can be traced back to AngularJS, released in 2010 (24), which revolutionized web development with its two-way data binding and extension to HTML vocabulary. AngularJS addressed the growing complexity of web applications by introducing an MVC (Model-View-Controller) architecture in the client-side application development process. Despite its success, the framework's limitations in terms of performance and mobile device support prompted a complete rewrite. So, in 2016 Angular 2 was launched, marking a significant departure from AngularJS. This new version adopted TypeScript, a superset of JavaScript offering optional static typing, classes, and interfaces. Angular's architecture was reimaged, focus next generation-based approach, which has since continued to evolve through versions, each adding improvements, new features, and refined practices such as Angular Ivy, a next-generation rendering engine designed for better performance and smaller bundle sizes.

Angular's history reflects not just the evolution of a framework but also the broader trends and shifts in web development practices, such as the move towards more modular, maintainable, and performance-optimized applications. Today, Angular stands as a robust, enterprise-level framework favored for its powerful features, extensive tooling, and strong community support, underlining its significance in the landscape of modern web development.

2.2.1 Core Concepts of Angular

The core concepts of Angular gives insight into the framework's comprehensive nature designed for building efficient and scalable web applications. Each of these core concepts plays a vital role in defining Angular's architecture, making it a robust framework for building complex applications. Angular's design philosophy emphasizes modularity, testability, and maintainability, facilitating the development of large-scale applications and improving productivity. Angular integrates robust tools and practices into its architecture, offering a wide array of functionalities to cater to complex development needs.

- **Modules:** Angular organizes applications into modules, with AppModule typically serving as the root module. Modules act as containers for different parts of an application, and they help in organizing the code, lazy-loading parts of the app, and bundling reusable code.

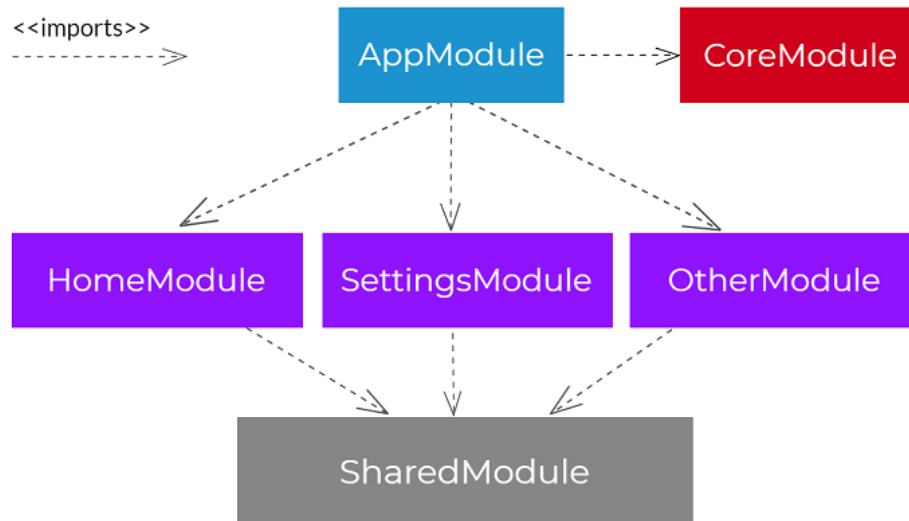


Figure 7. Modules in Angular (25)

- **Components:** At the heart of Angular applications are components, which control HTML views. Each component comprises an HTML template that specifies its rendering, a TypeScript class that outlines its behaviour, and metadata that dictates how the component is processed.

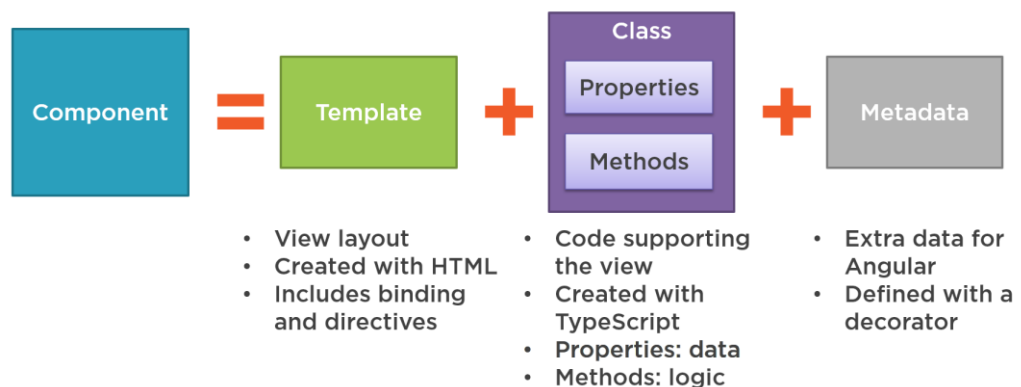


Figure 8. Components in Angular(26)

- **Templates:** In Angular, templates are developed using HTML that incorporates Angular-specific elements and attributes. These templates are designed to express the application's UI directly in HTML, augmented with Angular's data binding and directive capabilities.

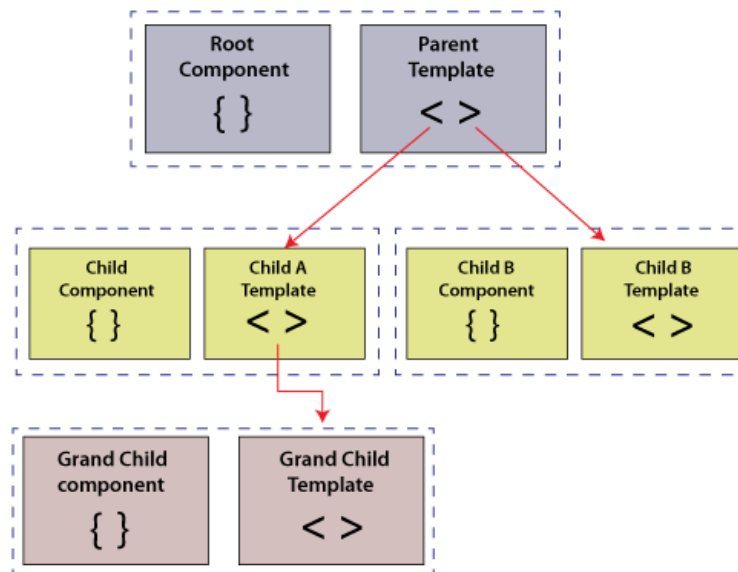


Figure 9. Templates in Angular (27)

- **Metadata:** Metadata decorates the class to define how it should be processed by Angular. It can associate a template with a component, provide information about an injectable class to Angular's dependency injection mechanism, or indicate how to compile a directive.

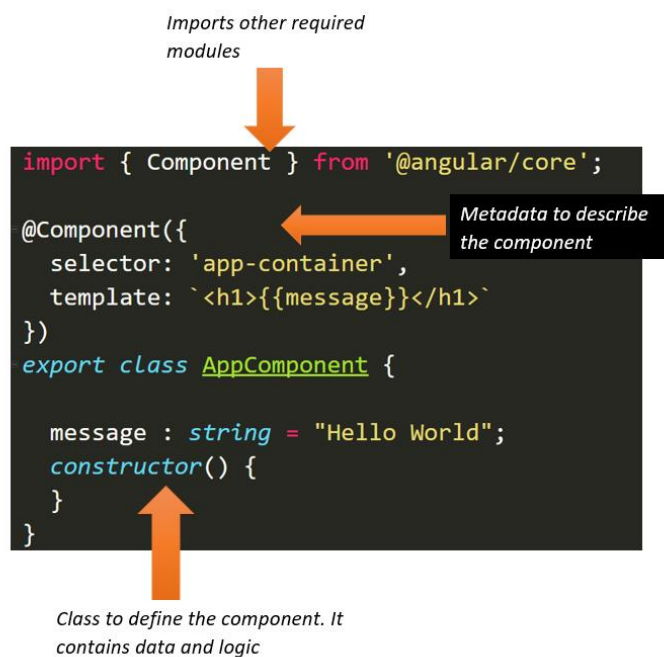


Figure 10. Metadata in Angular (28)

- **Data Binding:** Angular facilitates data binding, a technique for synchronizing sections of a template with sections of a component. Data binding can be either one-way (from component to view or view to component) or two-way, enabling interactive applications to swiftly respond to user inputs.

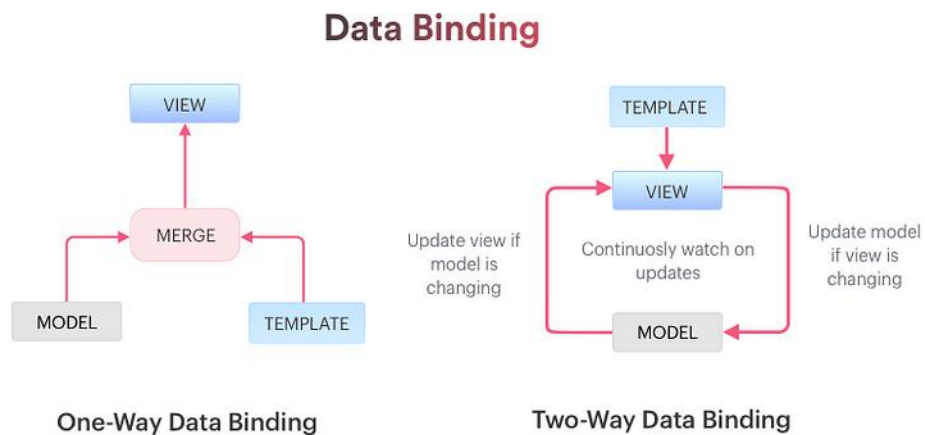


Figure 11. Data Binding in Angular (29)

- **Directives:** Angular's directives are functions that modify DOM elements. Structural directives have the capability to modify the DOM layout by adding or removing DOM elements, while attribute directives can alter the appearance or behaviour of an element, component, or another directive.

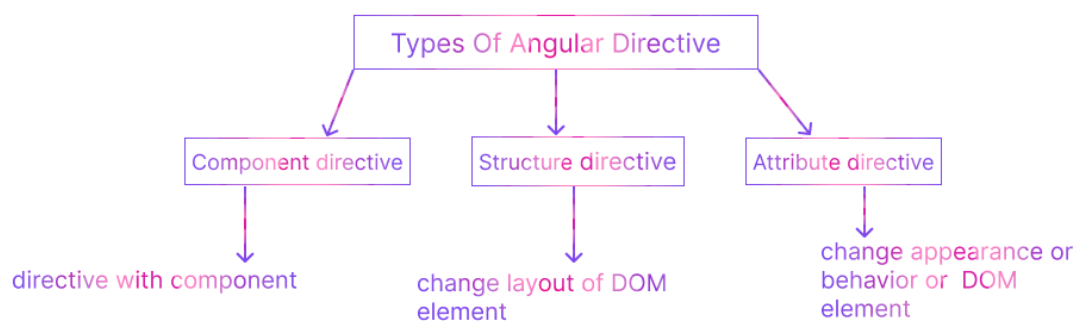


Figure 12: Directives in Angular

- **Services and Dependency Injection:** Services are broader scoped functions that perform specific tasks. Angular's dependency injection system permits the injection of services (and other values) into components, enhancing code modularity, reusability, and efficiency.

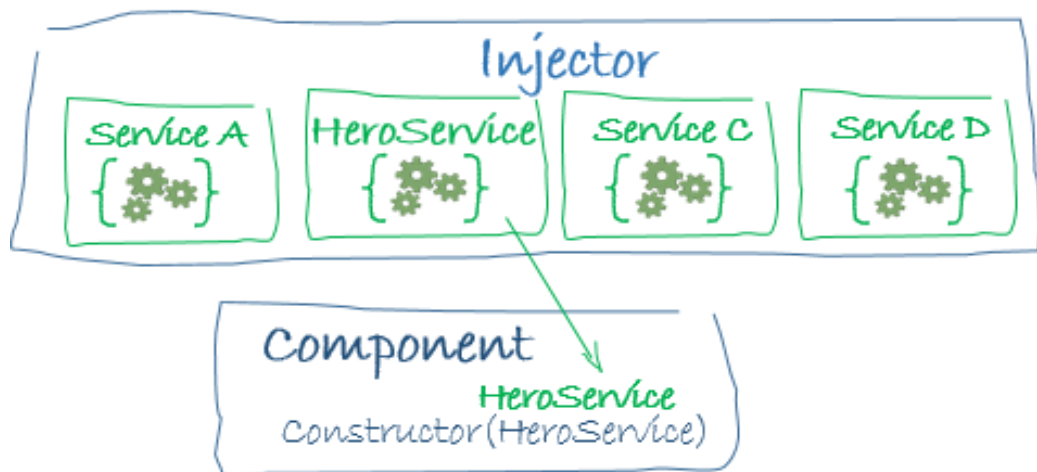


Figure 13. Services and Dependency Injection in Angular (30)

- Routing:** The Angular Router facilitates the transition between different views within an application as users engage in various tasks. Angular router is designed to integrate seamlessly with components, allowing developers to define navigation paths and view hierarchies that are intuitive and maintainable.

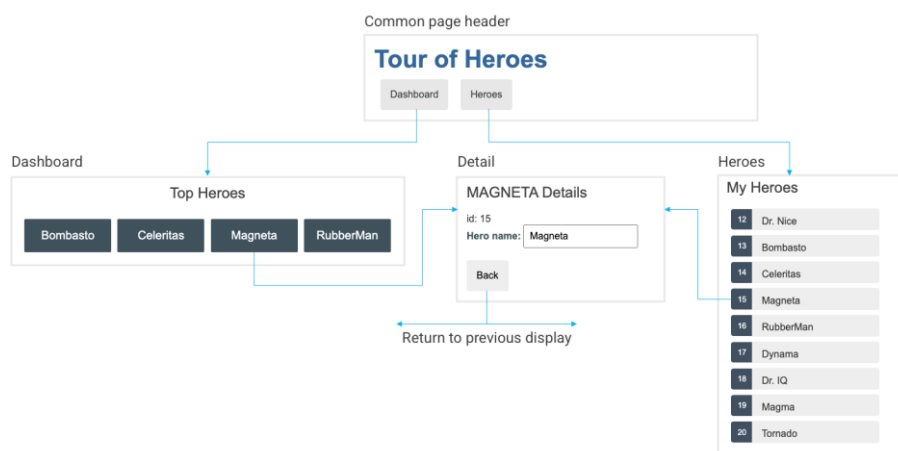


Figure 14. Routing in Angular (31)

- Angular CLI:** The Angular Command Line Interface (CLI) is an efficient tool designed to streamline the processes of initializing, developing, scaffolding, and maintaining Angular applications. It provides commands for project management, development tasks, testing, and bundling applications.

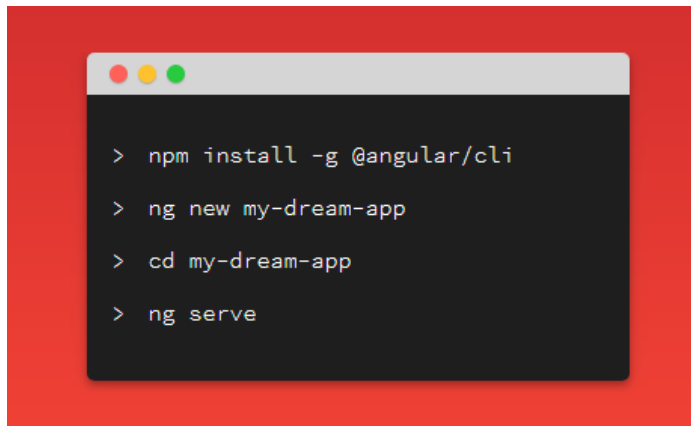


Figure 15. Angular CLI Tool (32)

2.2.2 Architecture

Angular's architecture is built on a modular structure that emphasizes maintainability, efficiency, and scalability, making it an excellent choice for developers creating complex applications. At the heart of Angular's design philosophy are NgModules, which segregate the application into cohesive blocks of functionalities. These modules make it possible to organize the codebase logically, facilitating features like lazy loading to improve application performance. Components are the primary building blocks within these modules, each responsible for rendering a part of the user interface and encapsulating specific logic. Angular enriches traditional HTML with its powerful template syntax, allowing dynamic content rendering and interactive user experiences through declarative UI definitions.

One distinctive feature of Angular's architecture is its use of TypeScript, which introduces strong typing, classes, and decorators, adding a layer of reliability and enhancing developer productivity through better tooling support. Metadata associated with classes, through decorators, instruct Angular on how to instantiate and use them, whether they represent components, services, or directives. Data binding in Angular is a seamless process, enabling a synchronized flow of information between the component's logic and the view, making state management more intuitive.

Directives further extend Angular's capabilities, allowing developers to manipulate DOM elements directly and build more dynamic, responsive applications. Services, another cornerstone of Angular's architecture, underscore the framework's commitment to reusability and modularity, providing

a systematic way to share functionality across components. The injection of these services, managed by Angular's Dependency Injection (DI) system, decouples components from their dependencies, enhancing testability and adherence to the SOLID (Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, and Dependency Inversion) principles of software design.

The Angular router supports the development of Single Page Applications (SPAs) by efficiently managing navigation paths and view hierarchies. This ensures users experience seamless interactions with various parts of the application without the need for full page reloads. This comprehensive architectural design, supported by the Angular CLI for project management and development tasks, underscores Angular's capability to tackle the challenges of modern web application development, offering a robust, integrated solution for building sophisticated applications.

2.2.3 Best suited use case

Angular's diverse ecosystem, combined with its design and development features, make it versatile for various web development projects. Whether it's building high-performance enterprise applications, engaging mobile apps, or interactive SPAs and PWAs, Angular offers a solid foundation, reducing development time and enhancing application quality and maintainability. Its comprehensive set of tools and features make it an ideal choice for several specific use cases such as:

Enterprise-level Applications: Angular's strong typing with TypeScript, cohesive package of solutions for various challenges, and scalable infrastructure fit the complex needs of large businesses and enterprises. The framework's architecture supports modularity, maintainability, and the efficient management of dependency injections, making it easier to develop, test, and deploy sizable applications with multiple teams involved.

Single-Page Applications (SPAs): SPAs benefit significantly from Angular's efficient update and rendering system, which minimizes page reloads for a smoother and more dynamic user experience. Angular's routing capabilities allow developers to control navigation flows within applications seamlessly, making it excellent for SPAs where user experience is paramount.

Mobile Applications: For projects aiming to extend their reach onto mobile platforms, When combined with frameworks such as NativeScript or Ionic, Angular offers a robust platform for developing cross-platform mobile applications from a single codebase. This capability allows businesses to maintain consistency across web and mobile platforms without sacrificing quality or performance.

Dynamic Content Applications: Applications that require frequent updates and dynamic content presentation, such as dashboards, data visualization tools, and content management systems, can

leverage Angular's databinding and directive features for real-time updates and interactivity, ensuring engaging user experiences.

Progressive Web Applications (PWAs): Angular provides built-in support for developing PWAs, which aim to combine the best of web and mobile app features. Its service worker management and application shell model enable the creation of fast, engaging, and reliable applications that can work offline and leverage the capabilities of modern browsers.

3 CRITERIA FOR RESEARCH

This section outlines the methodology and various aspects upon which the comparative study between React and Angular frameworks is based. The comparison aims to discern not only the technical capabilities of each framework but also their practical implications in real-world development scenarios.

3.1 Comparison criteria

Learning Curve: The learning curve for a new developer adopting React or Angular is a critical factor influencing their capacity to contribute effectively to a project. The learning curve will be evaluated, based on the onboarding experience for documentation, the intuitive nature of framework architectures, and the accessibility of the community for troubleshooting and learning resources.

Performance: Performance benchmarks are essential to understand how applications developed with React and Angular behave under various conditions. This criterion assesses the frameworks' efficiency in initial load times, indicating how swiftly an application becomes interactive to the user, and runtime efficiency, which examines the responsiveness and smoothness of applications during use. Performance comparisons can be quantitatively measured through metrics such as bundle sizes, time to interactive (TTI), and frames per second (FPS) in dynamic interactions.

Ecosystem and Tooling: A robust ecosystem and comprehensive tooling are indispensable for developer efficiency and application scalability. This entails the availability and maturity of libraries, extensions, build, and testing utilities surrounding React and Angular. The criterion examines how these factors contribute to solving common development challenges, supporting best practices, and integrating with other tools and services in the software development lifecycle (SDLC).

Community Support and Documentation: The vitality of a framework's community and the quality of its documentation significantly affect problem-solving efficiency and the learning curve. This comparison criterion explores the vibrant ecosystems of React and Angular—how responsive and helpful the communities are, the expansiveness, clarity, and update frequency of official documentation, and the presence of third-party resources and tutorials.

Popularity: Popularity reflects a framework's adoption rate, community size, and the overall sentiment within the developer ecosystem. It's an essential criterion as it influences the availability of learning resources, the ease of hiring skilled developers, and the general support structure for troubleshooting and enhancements. A popular framework often has a rich ecosystem, making it easier to find solutions to common challenges. High demand in the job market can be a proxy for the framework's longevity and industry relevance. In summary, popularity contributes to the framework's practical benefits beyond technical capabilities, affecting developers' learning curve, project support, and career prospects

3.2 Methodology

Sample Applications: Building a TO-DO List application in both React and Angular which serves as a hands-on approach to discern development nuances, identify unique challenges, and experience each framework's capabilities firsthand. This practical comparison allows for insights into setup processes, development workflows, and the out-of-the-box tooling each framework provides.

Performance Testing: Utilizing Chrome DevTools for performance benchmarking which ensures an objective methodology to measure and compare framework performance. Chrome DevTools offer detailed insights into load times, rendering performance, and resource utilization, facilitating a precise evaluation of how React and Angular applications perform under identical conditions. This approach not only anchors the comparison in tangible metrics but also reflects real-world application performance scenarios users might encounter.

3.2.1 Justification for Methodology

This methodology is designed to ensure a thorough and fair comparison between React and Angular. It combines theoretical exploration with practical experimentation and objective performance testing. By encompassing a range of factors from ease of learning to technical performance, and by adopting a hands-on approach to building and testing applications, this methodology seeks to offer a thorough understanding of the two frameworks. It enables a nuanced evaluation of each framework's strengths and weaknesses, thereby assisting developers, teams, and organizations in making well-informed decisions that align with their project requirements and objectives.

4 IMPLEMENTATION

4.1 Application Development

The practical implementation phase involved developing a Todo application using both React and Angular frameworks, with a common backend architecture. This process was instrumental in understanding the nuances, strengths, and limitations of each framework from a developer's perspective.

4.2 Backend Architecture

For both implementations, a Node.js server with Express.js was utilized, providing a consistent API layer. The database of choice was PostgreSQL, ensuring robust data persistence. However, the Object-Relational Mapping (ORM) tools differed between the two implementations:

- For the React backend: The 'pg' module was employed for direct PostgreSQL interaction.
- For the Angular backend: Sequelize ORM was implemented, offering an abstraction layer over database operations.

This distinction in ORM usage allowed for an evaluation of how each frontend framework interacts with different backend paradigms.

4.2.1 Core Functionalities

Both applications implemented the following key features:

1. User Authentication:
 - User registration
 - User login
 - Session management

2. Todo List Operations (CRUD):
 - Create new todo items
 - Read existing todo items
 - Update todo item status or content
 - Delete todo items

4.3 React Implementation

The development of the Todo application with React began with the setup of the development environment, leveraging Create React App for quick initialization and using Vite as the build tool and development server, which offers a modern and efficient development experience. The application's architecture adhered to React's component-based model, emphasizing modular design and reusability.

4.3.1 Architecture of React TodoApp:

This diagram below illustrates the three main layers of application:

1. Client Side (React Frontend):
 - Contains React components (Auth, ListHeader, ListItem, Modal, ProgressBar)
 - Communicates with the server via HTTP/HTTPS requests
2. Server Side (Express Backend):
 - Handles API routes (/todos, /signup, /login)
 - Processes requests and interacts with the database
3. Database (PostgreSQL):
 - Stores data in 'todos' and 'users' tables

The flow of data typically goes like this:

1. User interacts with React components
2. React makes API calls to the Express server
3. Express server processes the request, interacts with the PostgreSQL database
4. Database returns data to the server
5. Server sends a response back to the React frontend
6. React updates the UI based on the response Authentication Components (Login, Register)

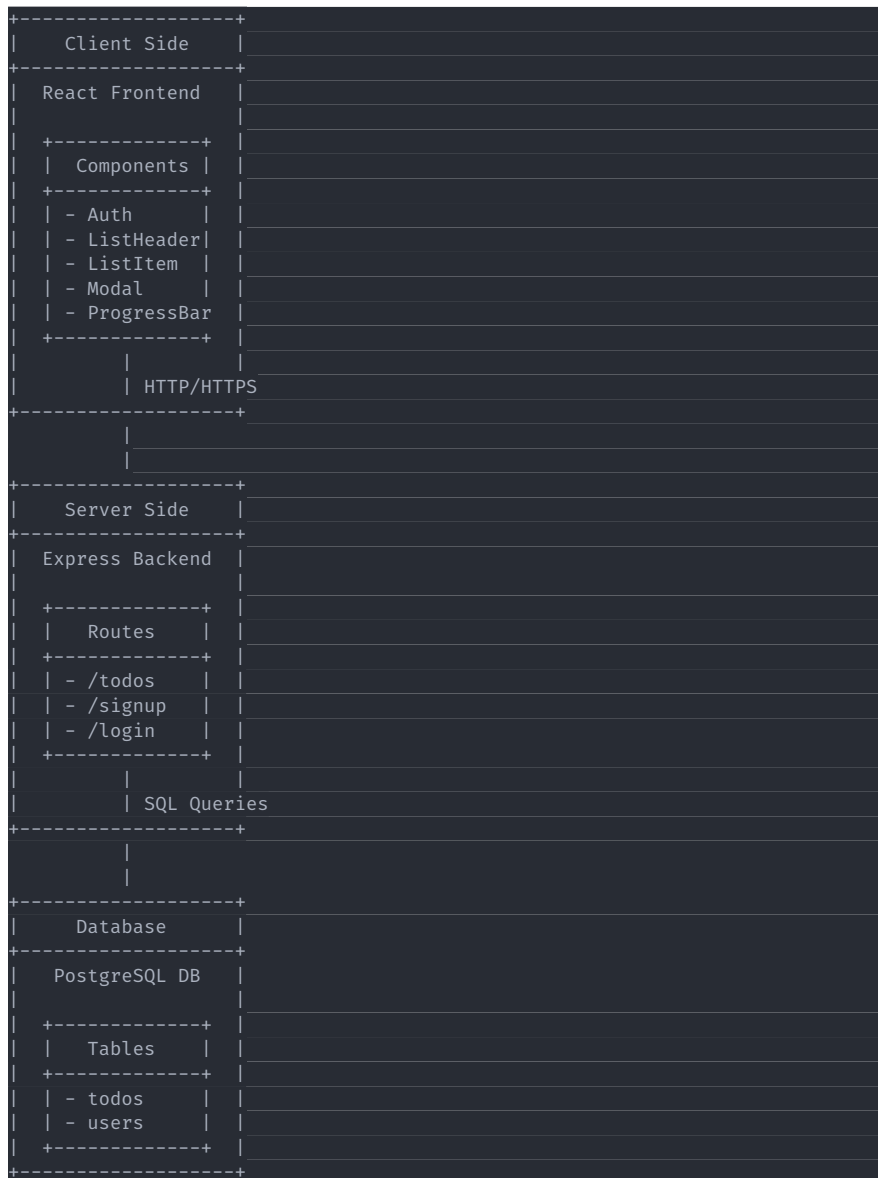


Figure 16. Architecture of React Todo App

4.3.2 Key Features

1. **State Management:** The application utilizes React's `useState` hook for local state management within components. For example, in the `App` component:

```
const [tasks, setTasks] = useState(null);
const [cookies, setCookie, removeCookie] = useCookies("user");
...
```

2. **Authentication:** The application implements a cookie-based authentication system using the `react-cookie` library:

```
const authToken = cookies.AuthToken;
const userEmail = cookies.Email;
```

3. **API Integration:** The application communicates with the backend API using the Fetch API. For instance, fetching todos:

```
const [tasks, setTasks] = useState(null);
const [cookies, setCookie, removeCookie] = useCookies("user");
...

const getData = async () => {
  try {
    const response = await fetch(
      `${import.meta.env.VITE_APP_SERVERURL}/todos/${userEmail}`
    );
    const json = await response.json();
    setTasks(json);
  } catch (err) {
    console.error(err);
  }
};
```

4. **Conditional Rendering:** React's declarative nature is leveraged for conditional rendering based on authentication status:

```
{!authToken && <Auth />}
{authToken && (
  <>
    <ListHeader listName={`My Todo Tasks`} getData={getData} />
    { /* ... */ }
  </>
)}
```

5. **Component Communication:** The application uses prop drilling for passing data and functions between components. For example, the `getData` function is passed down to child components for refreshing the todo list.

4.4 Angular Implementation

The Angular-based Todo application was developed using the Angular CLI, which provided a structured project scaffold and adhered to Angular's opinionated architecture. This implementation leveraged Angular's robust features for building scalable and maintainable single-page applications.

4.4.1 Architecture of Angular TodoApp

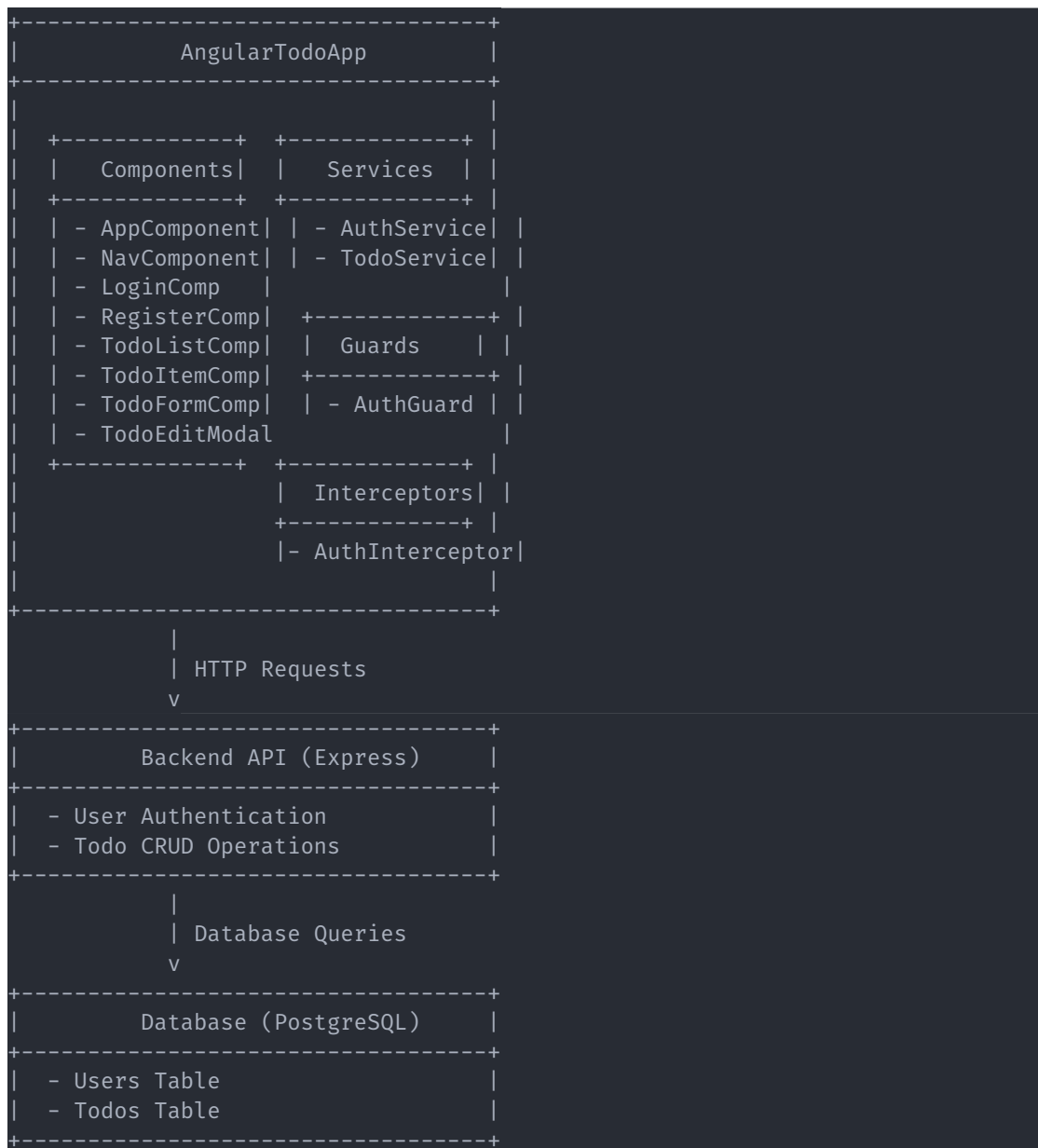


Figure 17. Architecture of Angular TodoApp

This diagram above illustrates the three main layers of Angular Todo App.

1. AngularTodoApp (Frontend):
 - **Components:** These are the building blocks of your UI. Each component handles a specific part of the application.
 - **Services:** These handle the business logic and data management. The AuthService manages authentication, while the TodoService handles todo-related operations.
 - **Guards:** The AuthGuard protects routes that require authentication.
 - **Interceptors:** The AuthInterceptor adds the authentication token to outgoing HTTP requests.
2. Backend API (Express):
 - Handles HTTP requests from the frontend.
 - Manages user authentication.
 - Performs CRUD operations on todos.
3. Database (PostgreSQL):
 - Stores user information and todos.

This architecture follows several key principles:

1. **Separation of Concerns:** The frontend is clearly separated from the backend, and within the frontend, components, services, and other features are modular.
2. **Single Responsibility:** Each component and service has a specific role. For example, the TodoService is responsible for all todo-related operations.
3. **Scalability:** This architecture can easily accommodate new features. For instance, if you want to add a new feature like todo categories, you can add a new component and service without significantly altering the existing structure.
4. **Security:** Authentication is handled by a dedicated service and guard, and the interceptor ensures that the auth token is included with requests.

4.4.2 Key Feature

1. **Dependency Injection:** Angular's dependency injection system is utilized for services and components. For example:

```
constructor(private todoService: TodoService) {}
```

2. **TypeScript Integration:** The application leverages TypeScript's strong typing for enhanced code quality and developer experience. For instance:

```
interface Todo {
  id: number;
  title: string;
  description: string;
  status: string;
}
```

3. **Reactive Programming:** Angular's integration with RxJS is used for handling asynchronous operations. For example, in the TodoService:

```
getAllTodos(): Observable<any> {
  return this.http.get(this.apiUrl);
}
```

4. **Template Syntax:** Angular's powerful template syntax is used for data binding and event handling:

```
<div *ngFor="let todo of todos">
  <app-todo-item
    [todo]="todo"
    (statusChanged)="updateTodoStatus($event)"
    (todoUpdated)="updateTodo($event)"
    (todoDeleted)="deleteTodo($event)"
  >
</app-todo-item>
</div>
```

5. **Decorators:** Angular's decorator pattern is used extensively for defining components, services, and other Angular-specific elements:

```
@Component({
  selector: 'app-todo-list',
  standalone: true,
  imports: [CommonModule, TodoFormComponent, TodoItemComponent],
  templateUrl: './todo-list.component.html',
  styleUrls: ['./todo-list.component.css'],
})
```

6. **API Integration:** The application is using Angular's HttpClient module for API communication. Services acted as intermediaries between components and the backend, encapsulating data fetching logic and promoting a clean separation of concerns.

Example of API integration in a service:

```

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment } from '../environments/environment';

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private apiUrl = environment.apiUrl;

  constructor(private http: HttpClient) { }

  login(email: string, password: string): Observable<any> {
    return this.http.post(`${this.apiUrl}/login`, { email, password });
  }

  signup(email: string, password: string): Observable<any> {
    return this.http.post(`${this.apiUrl}/signup`, { email, password });
  }
}

```

Figure 18. HttpClient module API integration in angular app

4.5 Comparative Observations

The implementation of the Todo application using both React and Angular frameworks revealed several notable distinctions and commonalities. This section presents a systematic comparison of the two approaches, focusing on key aspects of modern web application development.

4.5.1 Development Environment and Setup

React, utilizing Vite as the build tool with many other options to select from, offered rapid development cycles and efficient hot module replacement. The setup required minimal initial configuration, promoting quick project initiation. In contrast, Angular employed Angular CLI, providing a comprehensive project structure and built-in development server. It offered a more opinionated setup with

integrated testing tools and TypeScript configuration out of the box. While both environments facilitated efficient development, Angular's CLI provided a more structured initial setup, potentially beneficial for larger teams or projects requiring standardized architectures.

4.5.2 Component Architecture

React adopted a flexible, function-based component model using hooks for state management and side effects. This approach allowed for a more granular approach to component composition. Angular, on the other hand, implemented a class-based component structure with clear delineation between component logic and template. It utilized Angular's powerful dependency injection system for service integration. React's functional approach offered more flexibility in component design, while Angular's structured component architecture provided clearer separation of concerns, potentially aiding in long-term maintainability.

4.5.3 State Management

React primarily relied on React hooks (`useState`, `useEffect`) for local state management and utilized the Context API for global state when necessary. Angular leveraged RxJS observables for reactive state management and employed services as centralized state stores, facilitating a more structured approach to global state. Angular's use of RxJS provided a more robust solution for complex state scenarios, while React's hooks offered a simpler, more intuitive approach for less complex state management needs.

4.5.4 TypeScript Integration

In the provided React implementation, TypeScript was optional and not used. Type checking and IntelliSense support were limited to JSDoc comments or separate type declaration files. Angular, conversely, fully integrated TypeScript, providing enhanced type safety and tooling support throughout the application. It enforced strict typing, potentially reducing runtime errors and improving code quality. Angular's native TypeScript support offered stronger type checking and improved developer tooling, which could lead to fewer runtime errors and easier refactoring in large-scale applications.

4.5.5 API Integration

React utilized the native Fetch API for HTTP requests and implemented API calls directly within components or custom hooks. Angular employed the HttpClient module for API communication and encapsulated API logic within injectable services, promoting a clear separation of concerns. While both approaches effectively managed API integration, Angular's service-based approach provided a more structured and potentially more maintainable solution for complex applications with numerous API interactions.

4.5.6 Routing

React required an additional library (e.g., React Router) for routing capabilities, offering a more flexible, less opinionated routing solution. Angular provided a built-in routing module with powerful features like lazy loading and route guards, implementing a more structured, declarative routing approach. Angular's integrated routing solution offered more out-of-the-box functionality, while React's approach allowed for more customization but required additional setup.

4.5.7 Form Handling

React implemented forms using controlled components and React state, requiring manual implementation of form validation logic. Angular utilized reactive forms, providing a robust, model-driven approach to form management. It offered built-in form validation capabilities and custom validator support. Angular's reactive forms provided a more comprehensive solution for complex form scenarios, while React's approach offered simplicity for basic form implementations.

In conclusion, both React and Angular demonstrated their capabilities in effectively implementing the Todo List application. React excelled in its flexibility and simplicity, making it potentially more suitable for smaller projects or teams preferring a less opinionated framework. Angular, with its comprehensive feature set and structured approach, showed strengths in building large-scale, enterprise-level applications where consistency and robust tooling are paramount. The choice between these frameworks would ultimately depend on project requirements, team expertise, and long-term maintenance considerations.

Observation: Angular's reactive forms provided a more comprehensive solution for complex form scenarios, while React's approach offered simplicity for basic form implementations.

In conclusion, both React and Angular demonstrated their capabilities in effectively implementing the Todo List application. React excelled in its flexibility and simplicity, making it potentially more suitable for smaller projects or teams preferring a less opinionated framework. Angular, with its comprehensive feature set and structured approach, showed strengths in building large-scale, enterprise-level applications where consistency and robust tooling are paramount. The choice between these frameworks would ultimately depend on project requirements, team expertise, and long-term maintenance considerations.

5 ANALYSIS

This analysis section aims to provide a comprehensive comparison between React and Angular based on the implementation of a Todo application using both frameworks. The comparison criteria, as outlined in the methodology section, include:

1. Learning Curve
2. Performance
3. Ecosystem and Tooling
4. Community Support and Documentation
5. Popularity

The methodology employed a hands-on approach, developing identical Todo applications in both React and Angular. This practical implementation was supplemented by performance testing using Chrome DevTools and research into community metrics and developer surveys.

The findings will be presented systematically, addressing each criterion in turn. For each criterion, we will:

1. Provide observations from the Todo application implementation
2. Discuss general framework characteristics relevant to the criterion
3. Present quantitative data where applicable
4. Offer a comparative analysis between React and Angular

This structured approach aims to provide a clear, objective comparison while highlighting the practical implications of choosing either framework for web application development.

5.1 Learning Curve

The learning curve for both React and Angular is a crucial factor in their adoption and effectiveness in project development. This analysis compares the learning experiences of both frameworks based on the implementation of the Todo application and general framework characteristics.

5.1.1 Initial Setup Experience

Both React and Angular offer streamlined initial setup processes, but with some notable differences. The setup process using Create React App (CRA) was straightforward and quick. The command `npx create-react-app my-app` quickly scaffolds a new React project with a sensible default configuration. Similarly, Angular's CLI tool provides an easy setup with the command `ng new my-app`. This creates a new Angular project with a complete directory structure and necessary configuration files.

In both cases, the initial setup was relatively painless, allowing for a quick start to development. However, Angular's setup provides a more comprehensive project structure out of the box, which can be beneficial for larger projects but might feel overwhelming for simpler applications.

5.1.2 Complexity of Core Concepts

The core concepts of React and Angular differ significantly, impacting the learning curve:

React:

- JSX, while initially unfamiliar, proves intuitive for developers with HTML and JavaScript experience. It allows for a more natural blending of markup and logic.
- React's component-based architecture is straightforward, with a clear separation of concerns.
- Hooks (like `useState` and `useEffect`) provide a simple way to manage state and side effects in functional components.

Angular:

- TypeScript, while adding a layer of complexity, offers strong typing that can catch errors early and improve code quality. For developers with TypeScript experience, this is a significant advantage.
- Angular's architecture is more complex, involving concepts like modules, services, and dependency injection. These concepts, while powerful, require more time to fully grasp and utilize effectively.
- The framework's opinionated structure provides clear guidelines but can be overwhelming for newcomers.

From personal experience, having prior knowledge of React and TypeScript made certain aspects of Angular more approachable. However, Angular-specific concepts like modules, services, and dependency injection presented a steeper learning curve.

5.1.3 Time to Productivity

The time taken to become productive in each framework varied significantly. With prior experience, productivity in React was almost immediate. The familiarity with its concepts allowed for quick development of the Todo application. Despite previous TypeScript experience, reaching a comparable level of productivity in Angular took approximately two weeks. This time was necessary to understand and effectively implement Angular-specific concepts and best practices.

This difference in time to productivity highlights the more complex nature of Angular's architecture and the breadth of concepts it introduces.

5.1.4 Availability and Quality of Learning Resources

Both React and Angular benefit from extensive learning resources:

- **Documentation:** Both frameworks offer comprehensive official documentation. React's documentation is known for its clarity and beginner-friendly approach, while Angular's documentation is extensive and detailed, catering well to both beginners and advanced users.
- **Community Resources:** A wealth of community-created content exists for both frameworks, including blog posts, video tutorials, and courses. React, being more popular, has a slight edge in the sheer volume of available resources.
- **Example Projects:** Both frameworks have numerous example projects and code samples available. Angular's official documentation includes a comprehensive tutorial (Tour of Heroes) that covers many core concepts.
- **Video Tutorials:** Platforms like YouTube, Udemy, O'Reilly, Coursera and Frontend Masters offer extensive video courses for both React and Angular, catering to various skill levels.

The abundance and quality of learning resources for both frameworks are comparable, with slight differences in focus and approach. React's resources often emphasize quick starts and flexibility, while Angular's tend to be more structured and comprehensive.

5.1.5 Comparative Analysis

React's learning curve is generally gentler, allowing developers to start building applications quickly. Its flexibility and focus on JavaScript fundamentals make it accessible to a broader range of developers. However, this flexibility can also lead to decision fatigue and inconsistencies in large projects. Whereas Angular, with its more opinionated and comprehensive approach, presents a steeper initial learning curve. The need to understand TypeScript and Angular-specific concepts requires more upfront investment. However, once mastered, Angular's structured approach can lead to more consistent and maintainable code in large-scale applications.

The choice between React and Angular in terms of learning curve may depend on factors such as:

- The developer's prior experience (especially with TypeScript)
- The scale and complexity of the intended project
- The time available for learning and development
- The need for a highly structured approach versus flexibility

For quick prototypes or smaller projects, React's easier learning curve might be advantageous. For larger, enterprise-level applications where a structured approach is beneficial, the investment in learning Angular could pay off in the long term.

5.2 Performance Analysis

This performance analysis aims to provide a comprehensive comparison between React and Angular frameworks in the context of a Todo application implementation. The study focuses on key performance indicators that are crucial for web application efficiency and user experience. Our comparison will be based on the following metrics:

1. Initial Load Times: Evaluating how quickly each framework renders the initial content.
2. Runtime Performance: Assessing the responsiveness and smoothness of the application during use.
3. Operation-Specific Performance: Analyzing the efficiency of common CRUD (Create, Read, Update, Delete) operations.
4. Bundle Sizes: Comparing the total size of assets delivered to the client.
5. Code Efficiency: Examining how effectively each framework utilizes system resources.

6. Memory Usage: Measuring the memory footprint of each application.

Environment and Setup:

The analysis was conducted on the following environment:

- Operating System: Microsoft Windows 11 Pro
- System Type: x64-based PC
- Processor: Intel(R) Core (TM) i5-9300H CPU @ 2.40GHz, 2400 MHz, 4 Core(s), 8 Logical Processor(s)
- Installed Physical Memory (RAM): 16.0 GB
- Storage: Samsung SSD 980 500GB m.2
- Internet Connection: Download speed: 95.1 Mbps, Upload speed: 94.7 Mbps, Ping: 11ms, Jitter: 1ms
- Browser: Microsoft Edge for Business Version 127.0.2651.86 (Official build) (64-bit)

Data Collection Methods:

Chrome DevTools: Used for detailed performance profiling, including loading, scripting, rendering, painting, and system idle times.

Lighthouse: Employed for standardized web performance metrics such as First Contentful Paint, Largest Contentful Paint, and Total Blocking Time.

Built-in Developer Tools: Utilized for bundle size analysis and memory snapshots.

The React application was run on localhost:5173, connected to an Express server on localhost:8001, which communicated with a PostgreSQL database on localhost:5432. The Angular application was hosted on localhost:4200, with its Express server on localhost:8000, also connected to a PostgreSQL database on localhost:5432. For each operation, multiple iterations (around 5 iterations) were performed to account for variability, and average values were calculated. By analyzing these various aspects of performance across both React and Angular implementations, I aim to provide a comprehensive comparison that can guide developers in making informed decisions based on their specific project requirements and performance priorities.

This comprehensive approach allows for a holistic view of each framework's performance characteristics, providing valuable insights for developers and organizations considering either React or Angular for their web development projects.

5.2.1 Initial Load Times and Runtime Performance

Table 1. Lighthouse Performance Metrics (Average)

Metric	React (avg)	Angular (avg)	Difference(avg)
First Contentful Paint	0.96s	1.18s	+0.22S
Largest Contentful Paint	1.5s	3.5s	+2.08s
Total Blocking Time	14ms	78ms	+64ms
Cumulative Layout Shift	0.004	0	-0.004
Speed Index	1.02s	1.24s	+0.22s

The Lighthouse performance metrics provide insight into the initial load times and runtime performance of both React and Angular applications:

First Contentful Paint (FCP): React renders the first piece of DOM content 0.22 seconds faster than Angular, which is about 18.6% quicker. This suggests that users of the React application will see content slightly sooner, potentially improving perceived performance.

Largest Contentful Paint (LCP): React significantly outperforms Angular in this metric, with a 2.08 second (58%) faster LCP. This substantial difference indicates that React is much quicker in rendering the largest content element visible in the viewport, which is crucial for user perception of page load speed.

Total Blocking Time (TBT): React's TBT is 64ms shorter than Angular's, which is a 82% reduction. This suggests that the React application is more responsive to user interactions during the critical loading phase, potentially providing a smoother user experience.

Cumulative Layout Shift (CLS): Angular shows perfect stability with a CLS of 0, while React has a minimal shift of 0.004. Both values are excellent, indicating stable layouts during page load. Angular's perfect score might provide a slightly more stable visual experience, but the difference is negligible.

Speed Index: React's Speed Index is 0.22 seconds faster than Angular's, about 17.7% quicker. This indicates that the visible content of the React application populates more rapidly, contributing to a faster perceived load time.

React demonstrates superior performance across most metrics, particularly in areas crucial for user perception of speed and interactivity:

1. **Faster content rendering:** React's better FCP and significantly better LCP suggest that it provides content to users more quickly, which can be critical for user engagement and reducing bounce rates.
2. **Improved interactivity:** The substantially lower Total Blocking Time for React indicates that it becomes interactive more quickly, allowing users to interact with the page sooner.
3. **Quicker overall visual population:** React's better Speed Index suggests that it fills in visible content more rapidly, contributing to a perception of faster loading.

While Angular shows perfect layout stability (CLS of 0), React's performance is also very good in this aspect, and the difference is unlikely to be noticeable to users.

These metrics suggest that, in this implementation, React provides a faster and potentially more engaging initial user experience. The React application loads content more quickly, becomes interactive sooner, and generally provides a snappier first impression.

However, it's important to note that while these differences are significant, the actual impact on user experience can vary depending on the specific application, network conditions, and device capabilities. Both frameworks perform well in absolute terms, and the choice between them should also consider factors beyond just these initial load and runtime metrics, such as development efficiency, team expertise, and specific project requirements.

5.2.2 Operation-Specific Performance

Table 2. Average Operation Times (in milliseconds)

Operation	React (avg)	Angular (avg)	Difference(avg)
Loading Todos	13907.2	897.2	-13010.0
Creating Todo	11853.0	27938.8	+16085.8
Editing Todo	17150.6	22330.2	+5179.6
Deleting Todo	3564.2	4033.2	+469.0

The operation-specific performance data reveals interesting differences between React and Angular:

1. **Loading Todos:** Angular significantly outperforms React, completing the operation about 15.5 times faster. This substantial difference suggests that Angular may have more efficient data fetching and rendering mechanisms for initial data load.
2. **Creating Todo:** React shows superior performance in creating new todos, completing the operation in less than half the time of Angular. This indicates that React may handle state updates and DOM manipulation more efficiently for addition operations.
3. **Editing Todo:** React again demonstrates better performance, completing edit operations about 23% faster than Angular. This suggests React's virtual DOM and state management might be more optimized for update operations.
4. **Deleting Todo:** While React is still faster, the difference is less pronounced for deletion operations. React completes deletions about 11.6% faster than Angular, indicating that both frameworks handle removal operations relatively efficiently.

React shows superior performance in three out of four operations (creating, editing, and deleting), suggesting it may be more efficient for applications with frequent user interactions and data modifications. The speed advantage in these operations could lead to a more responsive feel in interactive parts of the application. However, Angular's dramatic performance advantage in loading todos is noteworthy. This could be particularly beneficial for applications that deal with large initial datasets or frequently reload data from the server. Angular's efficiency in data loading might provide a better user experience in data-heavy applications or those with less frequent user-driven data changes.

It's important to note that while these differences are significant, the actual impact on user experience would depend on the specific use case. For applications with smaller datasets or less frequent operations, the differences might not be noticeable to end-users. Conversely, for applications with large datasets or frequent operations, these performance differences could significantly impact the overall user experience.

Developers should consider these performance characteristics in the context of their specific application requirements. If the application primarily deals with displaying large amounts of initial data with infrequent updates, Angular's performance in loading todos might be more valuable. For applications with frequent user-driven changes, React's performance in creating, editing, and deleting operations could be more beneficial.

5.2.3 Bundle Size

Table 3. Bundle Size and Load Time Comparison

Metric	React	Angular	Difference
Requests	25	45	+20
Transferred	661 kB	745 kB	+84 kB
Resources	2.0 MB	4.2 MB	+2.2 MB
Finish Time	369ms	805ms	+436ms
DomContentLoaded	167ms	517ms	+350ms
Load	316ms	733ms	+417

The bundle size and load time metrics reveal significant differences between React and Angular:

1. Number of Requests: Angular requires 80% more requests than React, which could potentially lead to increased network overhead.
2. Transferred Data: Angular transfers about 12.7% more data than React, which is a relatively small difference compared to other metrics.
3. Resources: Angular's total resource size is more than double that of React (4.2 MB vs 2.0 MB). This substantial difference could impact load times and performance, especially on slower networks or devices with limited resources.
4. Load Times:
 - Finish Time: Angular takes 118% longer to finish loading compared to React.
 - DOMContentLoaded: Angular's DOMContentLoaded event fires 209% later than React's.
 - Load Time: Angular's full page load is 132% slower than React's.

React demonstrates superior performance across all bundle size and load time metrics. Its smaller resource size, fewer requests, and faster load times suggest that React applications may provide a more responsive user experience, particularly on slower networks or less powerful devices. The significantly faster DOMContentLoaded time for React indicates that it allows for quicker user interaction with the page content.

While Angular's larger bundle size and longer load times don't necessarily preclude its use, they do suggest that more attention may need to be paid to optimization techniques such as lazy loading and code splitting when working with Angular to achieve comparable load performance to React.

5.2.4 Code Efficiency

Table 4. Code Usage Efficiency

Metric	React	Angular
Total Code Size	1.4 MB	3.4 MB
Used Code	558 KB	2.1 MB
Unused Code	788 KB	1.4 MB
Code utilization Rate	42.6%	60.0%

Total Code Size: Angular's codebase is significantly larger, about 2.4 times the size of React's. This larger codebase could potentially lead to longer load times and higher memory usage.

Used Code: Angular uses more code in absolute terms (2.1 MB vs 585 KB). This suggests that Angular might be providing more out-of-the-box functionality, but it also means it's loading more code into memory.

Unused Code: Both frameworks have substantial amounts of unused code, with React having 788 KB and Angular 1.4 MB. This unused code represents potential for optimization in both frameworks.

Code Utilization Rate: Angular shows a higher code utilization rate at 60%, compared to React's 42.6%. This indicates that despite its larger codebase, Angular is using a greater proportion of its code.

Detailed Code Breakdown for React:

node_modules/.vite/deps/react.js?v=47ddd095	JS (per function) 155 00%
node_modules/vite/dist/client/env.mjs	JS (per function) 4,021 30 67.6%
src/App.css	JS (per function) 16,106 3 70.2%
src/App.jsx	JS (per function) 11,307 24 92.2%
src/components/Auth.jsx	JS (per function) 17,202 5,741 33.4%
src/components/checkmarkIcon.jsx	JS (per function) 5,225 5,017 96%
src/components/ListHeader.jsx	JS (per function) 7,591 37 44.9%
src/components/ListItem.jsx	JS (per function) 9,960 62 96.3%
src/components/Modal.jsx	JS (per function) 17,201 5,685 33.1%
src/components/ProgressBar.jsx	JS (per function) 5,169 24 94.8%
src/components/TickIcon.jsx	JS (per function) 6,202 24 94%
src/index.css	JS (per function) 1,711 3 72.2%
src/main.jsx	JS (per function) 2,154 00%

Detailed Code Breakdown for Angular:

@angular_platform-browser_animations_async.js	JS (per function)	8,224	2,317	28.2%
css		3,660	2,010	54.9%
chunk-HZE6N6ST.js	JS (per function)	796	367	46.1%
env.mjs	JS (per function)	4,021	306	7.6%
rxjs.js	JS (per function)	4,966		00%
rxjs_operators.js	JS (per function)	3,269		00%
@angular_platform-browser.js	JS (per function)	2,774		00%
@angular_material_form-field.js	JS (per function)	1,820		00%
@angular_material_core.js	JS (per function)	2,708		00%
@angular_forms.js	JS (per function)	3,388		00%
@angular_core.js	JS (per function)	23,471		00%
@angular_common.js	JS (per function)	4,644		00%
@angular_common_http.js	JS (per function)	2,610		00%

Key Observations:

React Components:

- Most React components show high utilization rates (>90%), indicating efficient use of component code.
- The Auth and Modal components have lower utilization (33.4% and 33.1% respectively), suggesting potential for optimization.

Angular Modules:

- Many Angular modules show 0% utilization, which could indicate that these modules are loaded but not used in the current application state.
- The platform-browser animations module shows 28.2% utilization, suggesting that not all animation capabilities are being used.

Framework Core:

- React's core library (react.js) shows 0% utilization in this snapshot, which is unusual and might indicate a measurement anomaly.
- Angular's core library (@angular_core.js) also shows 0% utilization, which is similarly unexpected and may be due to how the usage is measured.

Build Tools:

- Both frameworks show relatively low utilization of their build tool code (Vite for React, chunk-HZE6N6ST.js for Angular), which is normal as much of this code is for development purposes.

While Angular has a larger codebase, it demonstrates better overall code utilization. This suggests that Angular's larger size is partly justified by its use of more of its included functionality. However, there's significant room for optimization in both frameworks, particularly in reducing unused code.

React's smaller codebase and high component utilization rates suggest it might be more suitable for smaller, focused applications. Angular's broader feature set and higher overall utilization might make it more appropriate for larger, feature-rich applications where more of its built-in capabilities are likely to be used.

Both frameworks show opportunities for optimization, particularly in reducing unused code and potentially lazy-loading certain modules or components. This could lead to improved performance and reduced resource usage in both React and Angular applications.

5.2.5 Memory Usage

Table 5. Memory Usage Comparison (in KB).

Memory Type	React	Angular	Difference
Code	5,648	17,499	+11,851
Strings	1,174	1,870	+696
JS Array	131	975	+844
Typed Array	297	833	+731
System Array	437	1168	731
Total	7,687	22,345	14,5658

1. **Code Memory:** Angular uses significantly more memory for code, about 3.1 times more than React. This aligns with our previous observations about Angular's larger codebase. The substantial difference (11,851 KB) suggests that Angular loads more features and functionalities into memory, which could be beneficial for complex applications but may impact performance on memory-constrained devices.
2. **Strings:** Angular uses 59.3% more memory for strings compared to React. This could indicate more text content, templates, or string-based configurations in the Angular application.
3. **JS Array:** Angular's JS array memory usage is dramatically higher, using 7.4 times more memory than React. This substantial difference might be due to Angular's more extensive use of arrays for data management, possibly related to its change detection mechanism or component tree structure.

4. **Typed Array:** Angular uses 2.8 times more memory for typed arrays. This could be related to Angular's use of TypeScript and its more rigid typing system, potentially storing more type of information in memory.
5. **System Objects:** Angular consumes 2.7 times more memory for system objects. This higher usage might be due to Angular's more complex internal object structure, possibly related to its dependency injection system or component lifecycle management.
6. **Total Memory Usage:** Overall, the Angular application uses 2.9 times more memory than the React application. This significant difference (14,658 KB or about 14.3 MB) could have notable impacts on performance, especially on devices with limited resources.

Key Observations:

1. **Overall Efficiency:** React demonstrates superior memory efficiency across all categories. Its lower memory footprint could lead to better performance, especially on mobile devices or in memory-constrained environments.
2. **Framework Architecture Impact:** The substantial difference in code memory usage (11,851 KB) reflects the architectural differences between React and Angular. Angular's higher memory usage is likely due to its more comprehensive, "batteries-included" approach, which loads more features upfront.
3. **Data Management:** The significant difference in JS Array memory usage suggests that Angular and React have very different approaches to managing data structures in memory. React's approach appears to be more memory-efficient in this implementation.
4. **Scalability Considerations:** While React's lower memory usage is advantageous, it's important to consider how these numbers might scale with application complexity. Angular's higher baseline might be offset by more gradual increases as the application grows.
5. **Performance Implications:** The lower memory usage in React could translate to faster application performance, particularly in areas like garbage collection and overall system responsiveness.

React demonstrates significantly better memory efficiency in this implementation, using less than half the memory of Angular across all categories. This efficiency could translate to better performance, particularly on devices with limited resources or for applications that need to maintain a small memory footprint.

Angular's higher memory usage, while potentially impacting performance, may be justified by its more comprehensive feature set. The additional memory could be supporting more out-of-the-box functionalities that might require extra setup or third-party libraries in React.

For developers and organizations choosing between these frameworks, these memory usage statistics should be considered alongside other factors:

1. If memory efficiency and performance on lower-end devices are critical, React might be the better choice.
2. If the application requires many of Angular's built-in features and the target environment has sufficient resources, Angular's higher memory usage might be acceptable.
3. For large-scale applications, it would be important to profile memory usage as the application grows to understand how these baseline differences scale with increased complexity.

Ultimately, the choice should balance these memory considerations with other factors like development speed, team expertise, and specific project requirements.

5.2.6 Conclusion:

This comprehensive performance analysis of React and Angular reveals distinct strengths and characteristics for each framework:

1. **Initial Load Times and Runtime Performance:** React consistently outperforms Angular in key metrics such as First Contentful Paint, Largest Contentful Paint, and Total Blocking Time. This suggests that React applications may provide a faster and more responsive initial user experience.
2. **Operation-Specific Performance:** React excels in creating, editing, and deleting operations, while Angular shows superior performance in loading todos. This indicates that React might be more suitable for applications with frequent user interactions, while Angular could be advantageous for data-heavy applications with large initial datasets.
3. **Bundle Sizes and Load Times:** React demonstrates smaller bundle sizes and faster load times across all metrics. This efficiency in resource utilization could lead to better performance, especially on slower networks or less powerful devices.
4. **Code Efficiency:** While Angular has a larger codebase, it shows a higher code utilization rate (60% vs React's 42.6%). This suggests that Angular's larger size is partly justified by its use of more included functionality. However, both frameworks show significant room for optimization in reducing unused code.

- 5. Memory Usage:** React exhibits substantially lower memory usage across all categories, using less than half the memory of Angular. This efficiency could translate to better performance, particularly on devices with limited resources.

The choice between React and Angular should be based on specific project requirements and constraints:

- For projects prioritizing fast initial load times, smaller bundle sizes, and efficient memory usage, especially on resource-constrained devices or slower networks, React appears to be the more suitable choice.
- For large-scale, feature-rich applications that can benefit from Angular's comprehensive built-in functionality and don't have strict resource constraints, Angular could be more appropriate. Its better performance in loading large datasets could be advantageous for data-intensive applications.
- Development teams should also consider factors beyond pure performance metrics, such as:
 - The learning curve and existing team expertise
 - Long-term maintenance requirements
 - Specific feature needs that might be better served by one framework's ecosystem
 - Scalability requirements of the project

Both frameworks have shown they can be optimized for better performance. Regardless of the chosen framework, implementing best practices such as code splitting, lazy loading, and regular performance audits will be crucial for maintaining optimal application performance.

Ultimately, while this analysis provides valuable insights into the performance characteristics of React and Angular, the best choice will depend on a holistic evaluation of project requirements, team capabilities, and the specific use case of the application being developed.

5.3 Ecosystem and Tooling

The ecosystem and tooling surrounding a framework play a crucial role in its adoption, development efficiency, and long-term maintainability. Both React and Angular have mature ecosystems, but they differ significantly in their approach and composition.

5.3.1 React Ecosystem and Tooling:

React's ecosystem is characterized by its flexibility and community-driven nature. This approach allows developers to choose from a wide array of tools and libraries, tailoring their development environment to specific project needs.

1. **Package Management:** React primarily uses npm or Yarn for package management. These tools provide access to a vast registry of JavaScript packages, including React-specific libraries.
2. **Build Tools:** While Create React App offers a zero-configuration setup, many developers opt for more customizable build tools like Webpack or Vite. Vite, in particular, has gained popularity for its faster build times and hot module replacement.
3. **State Management:** React's ecosystem offers multiple state management solutions. Redux has been a popular choice for large-scale applications, while newer alternatives like MobX and Recoil provide different approaches to state management.
4. **Routing:** React Router is the de facto standard for routing in React applications, offering a declarative approach to navigation.
5. **UI Component Libraries:** The React ecosystem boasts a plethora of UI component libraries such as Material-UI, Ant Design, and Chakra UI, providing pre-built, customizable components.
6. **Testing:** Jest, developed by Facebook, is commonly used for unit and integration testing in React applications. Tools like React Testing Library facilitate component testing.
7. **Developer Tools:** React Developer Tools, a browser extension, aids in debugging and inspecting React component hierarchies.

5.3.2 Angular Ecosystem and Tooling:

Angular's ecosystem is more opinionated and integrated, providing a comprehensive suite of tools and libraries designed to work seamlessly together.

1. **Angular CLI:** This command-line interface tool is central to Angular development, offering commands for project scaffolding, development, testing, and deployment.
2. **Package Management:** Angular uses npm for package management, with a focus on packages specifically designed for the Angular ecosystem.
3. **Build Tools:** Angular CLI integrates Webpack for bundling and optimization, abstracting much of the configuration complexity.
4. **State Management:** NgRx, an RxJS-powered state management library, is commonly used for large-scale Angular applications. It implements the Redux pattern but is specifically designed for Angular.
5. **Routing:** Angular provides a built-in router module, offering features like lazy loading and route guards out of the box.
6. **UI Component Libraries:** Angular Material, developed by the Angular team, provides a comprehensive set of pre-built, Material Design-based components. Other options like PrimeNG and NGX-Bootstrap are also popular.
7. **Testing:** Angular comes with integrated testing tools, including Karma for unit testing and Protractor for end-to-end testing. Jasmine is the default testing framework.
8. **Developer Tools:** Angular DevTools, available as a browser extension, assists in debugging and profiling Angular applications.

5.3.3 Comparative Analysis:

1. **Integration:** Angular's ecosystem is more tightly integrated, providing a cohesive development experience. React's ecosystem, while more flexible, may require additional setup and integration efforts.
2. **Learning Curve:** Angular's comprehensive tooling may present a steeper initial learning curve, but it can lead to more consistent development practices across teams. React's flexible ecosystem allows for a more gradual adoption of tools as needed.

3. **Customization:** React's ecosystem offers more flexibility in choosing and replacing individual tools. Angular's integrated approach provides less flexibility but ensures better compatibility between tools.
4. **Enterprise Readiness:** Angular's opinionated ecosystem and built-in tools make it well-suited for enterprise-level applications where consistency and standardization are crucial. React's flexibility can be advantageous for startups and projects with unique requirements.
5. **Community Support:** Both ecosystems benefit from strong community support, but React's larger and more diverse ecosystem may offer more community-driven solutions for specific needs.

In conclusion, the choice between React and Angular's ecosystems largely depends on project requirements and team preferences. React's ecosystem favors flexibility and customization, allowing developers to tailor their toolchain to specific needs. Angular's ecosystem, on the other hand, offers a more integrated and opinionated approach, which can be beneficial for large-scale applications and teams seeking a standardized development environment.

It's important to note that this analysis is based on the current state of these ecosystems as of July 2024. The rapidly evolving nature of web development means that new tools and trends may emerge, potentially altering the landscape of these ecosystems.

5.4 Community Support and Documentation Analysis

The strength of a framework's community support and the quality of its documentation are crucial factors in its adoption, ease of use, and long-term sustainability. This analysis examines how React and Angular fare in these aspects, considering the breadth and depth of their community engagement and the comprehensiveness of their official documentation.

5.4.1 React Community Support and Documentation:

1. **Community Size and Activity:** React boasts a large and active community. As of April 2024, React's GitHub repository has over 200,000 stars and more than 1,600 contributors. The high level of community engagement is evident in the frequent discussions on platforms like Stack Overflow, Reddit's `r/reactjs` subreddit, and various Discord channels.

2. **Third-party Resources:** The React ecosystem is rich with third-party tutorials, courses, and books. Platforms like Udemy, Coursera, and egghead.io offer numerous React courses. This abundance of resources caters to developers at various skill levels, from beginners to advanced practitioners.
3. **Official Documentation:** React's official documentation is known for its clarity and comprehensive coverage of core concepts. It includes:
 - A "Quick Start" guide for beginners
 - Detailed API references
 - Advanced guides on topics like code-splitting and context
 - A tutorial section for hands-on learning

The documentation is regularly updated to reflect new features and best practices. However, due to React's flexible nature, developers often need to consult additional resources for topics like state management or routing, which are not part of React's core library.

4. **Community-driven Libraries:** React's philosophy of being a UI library rather than a full-fledged framework has led to a proliferation of community-driven libraries. While this offers flexibility, it can sometimes lead to decision fatigue for developers when choosing the right tools for their projects.

5.4.2 Angular Community Support and Documentation:

1. **Community Size and Activity:** Angular has a substantial community, though slightly smaller than React's. As of April 2024, Angular's GitHub repository has over 85,000 stars and more than 1,500 contributors. The community is active on platforms like Stack Overflow, the Angular subreddit, and Gitter chat rooms.
2. **Third-party Resources:** Angular has a strong presence in professional training resources. Platforms like Pluralsight and LinkedIn Learning offer comprehensive Angular courses. The framework's popularity in enterprise environments has led to a wealth of resources tailored for professional development teams.
3. **Official Documentation:** Angular's documentation is known for its thoroughness and structured approach. It includes:
 - A comprehensive tutorial (Tour of Heroes)
 - Detailed guides on core concepts and best practices

- API references
- CLI documentation
- Cookbook recipes for common scenarios

The documentation is well-organized and regularly updated. It covers not just core Angular concepts but also related topics like RxJS and NgRx, providing a more holistic learning resource.

4. Angular-specific Conferences and Events: The Angular community benefits from dedicated conferences like NG-Conf and Angular Connect, which foster knowledge sharing and community building.

5.4.3 Comparative Analysis:

1. Community Engagement: Both React and Angular have highly engaged communities. React's larger community size can lead to faster problem-solving and a wider variety of community-created tools. Angular's community, while smaller, tends to be more focused and often includes more enterprise developers.
2. Learning Resources: React's flexibility has resulted in a wider array of learning resources, catering to various development styles. Angular's resources are more standardized, often following similar patterns and best practices across different sources.
3. Documentation Quality: Angular's documentation is more comprehensive and self-contained, covering most aspects of application development within its ecosystem. React's documentation, while excellent for core concepts, often requires developers to seek additional resources for a complete application stack.
4. Ecosystem Stability: Angular's more opinionated nature and comprehensive documentation can lead to a more stable ecosystem, with clearer upgrade paths and best practices. React's ecosystem, while more diverse, can sometimes suffer from rapid changes and conflicting approaches.
5. Corporate Backing: Both frameworks benefit from strong corporate backing (Facebook for React, Google for Angular), which ensures continued development and support. This backing translates into well-maintained documentation and regular updates.

In conclusion, both React and Angular offer strong community support and high-quality documentation, but with different emphases. React's community is larger and more diverse, offering a wide range of solutions and approaches, which can be both a strength and a challenge for developers. Angular's community and documentation provide a more structured and comprehensive resource set, which can be particularly beneficial for large-scale, enterprise applications. It's important to note that community dynamics and documentation quality can evolve over time. This analysis is based on the state of these aspects as of July 2024, and future developments may alter this landscape.

5.5 Popularity Analysis

Popularity in the context of front-end frameworks is a multifaceted metric that encompasses adoption rates, community engagement, and developer sentiment. It serves as a crucial indicator of a framework's ecosystem health, resource availability, and potential longevity. This analysis examines the popularity of React and Angular through various lenses, utilizing data from multiple reputable sources.

5.5.1 Developer Surveys and Usage Statistics:

JetBrains Developer Ecosystem Survey 2023: According to the JetBrains survey (33), React leads in popularity among JavaScript frameworks, with 57% of respondents using it. Angular follows at 20%, indicating a significant gap in adoption rates.

JavaScript frameworks and libraries

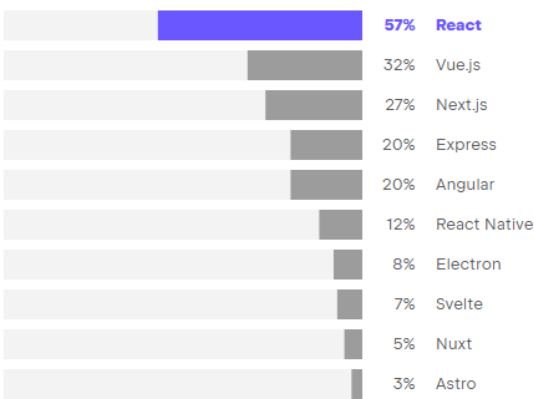


Figure 19. JetBrains DE ecosystem report 2023 for javascript

Stack Overflow Developer Survey 2023: The Stack Overflow survey(34) corroborates React's dominance, with 40.58% of professional developers reporting React usage. Angular shows a usage of 17.46%, further reinforcing the popularity gap between the two frameworks.



Figure 20. stack overflow most popular web framework survey(34)

State of JavaScript 2023: The State of JavaScript report(35) provides insights into developer satisfaction and usage. React maintains a high satisfaction rate of 71.2% and a usage rate of 84%. Angular, while still widely used (45%), shows a lower satisfaction rate of 32.5%, potentially impacting its adoption trends.

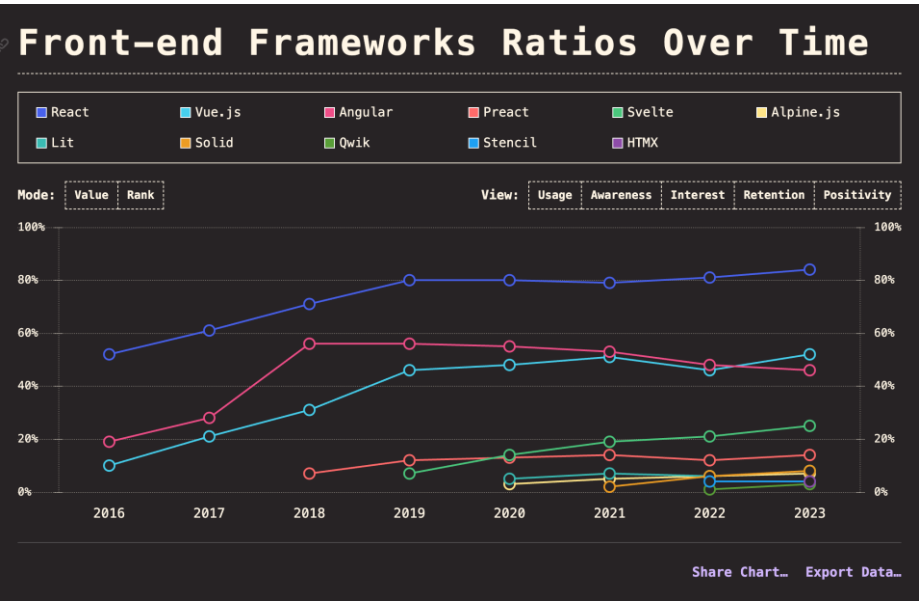


Figure 21. frontend framework usage overt time sateofjs.2023(35)

5.5.2 Google Trends Analysis:

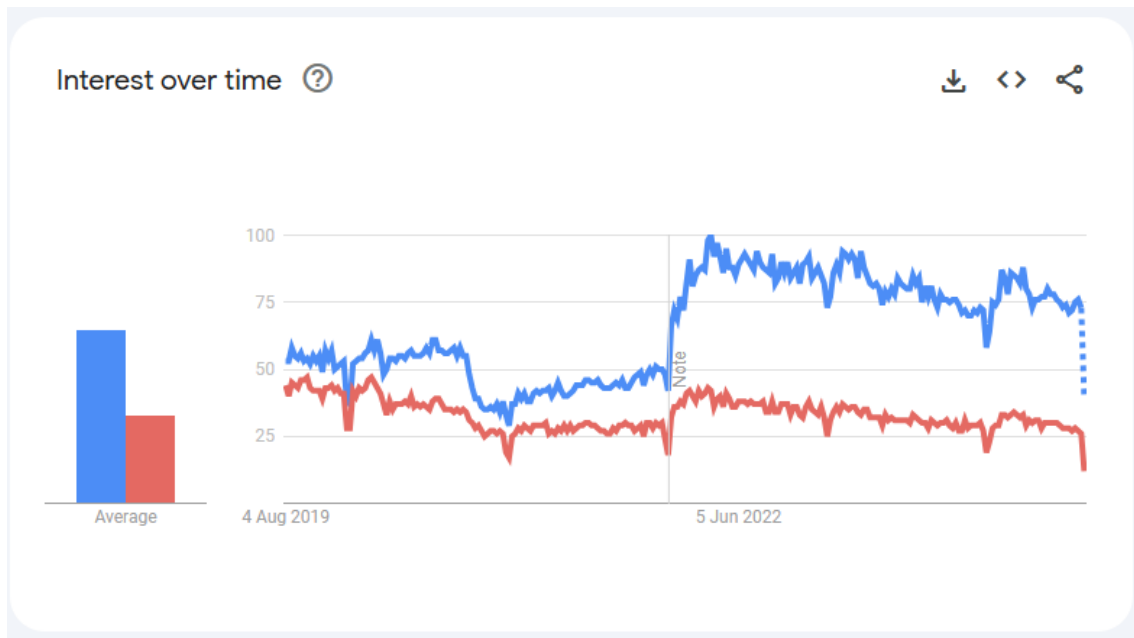


Figure 22. google trends - web search for react vs angular in past 5 years(36)

Examining Google Trends data for the past five years reveals: React consistently maintains higher search interest compared to Angular. Both frameworks show relative stability in search trends, indicating sustained interest. React's search volume is approximately 1.5 to 2 times higher than Angular's throughout the period.

5.5.3 Website Usage Statistics:

BuiltWith technology(37) usage statistics provide insights into real-world adoption:

As of July 2024, React is used by 48,969,543 live websites, including 18.91% of the top 1 million websites(38). Angular (considering both AngularJS and Angular 2+) is used by 1,273,738 live websites, with 7.3% usage among the top 1 million websites. These figures demonstrate React's significantly broader adoption in production environments.

5.5.4 Comparative Analysis:

Adoption Trends: React consistently outperforms Angular in terms of adoption across various surveys and usage statistics. This widespread adoption contributes to a larger ecosystem of tools, libraries, and resources for React developers.

Developer Satisfaction: React shows higher developer satisfaction rates, which can be a leading indicator of future adoption trends. Angular's lower satisfaction scores may impact its long-term popularity, although its strong presence in enterprise environments provides a stable user base.

Learning Resources and Community Support: React's higher popularity translates to a more extensive range of learning resources, from tutorials and courses to community-driven documentation. This abundance of resources can lower the entry barrier for new developers.

Job Market Implications: The higher adoption rate of React suggests a potentially larger job market for React developers. However, Angular's strong enterprise presence ensures a steady demand for Angular skills, particularly in larger organizations.

Ecosystem Stability: While React's larger ecosystem offers more choices, it can also lead to decision fatigue and potential instability as libraries evolve. Angular's more opinionated nature and slower adoption of new trends can provide a more stable ecosystem, which is particularly valuable in enterprise settings.

5.5.5 Conclusion:

The popularity analysis clearly indicates React's dominance in the front-end framework landscape. Its higher adoption rates, developer satisfaction, and usage in production environments suggest a robust ecosystem with abundant resources and community support. However, Angular maintains a significant market share, particularly in enterprise environments, ensuring its continued relevance.

It's important to note that popularity should not be the sole criterion for choosing a framework. Project requirements, team expertise, and long-term maintenance considerations should also play crucial roles in the decision-making process. Furthermore, the front-end landscape is dynamic, and these trends may evolve over time.

This analysis is based on data available as of July 2024, and future surveys or significant technological shifts could alter the popularity landscape of these frameworks.

5.6 Limitations of the scope of Analysis

While this comparative study of React and Angular strives for comprehensiveness and objectivity, it is important to acknowledge several limitations that may impact the interpretation and generalizability of the findings:

- **Temporal Constraints:** The rapidly evolving nature of web development technologies means that the landscape can change quickly. This analysis is based on data and observations as of April 2024. Future updates to either framework or shifts in development practices could alter the validity of some conclusions.
- **Scope of Implementation:** The practical implementation focused on a Todo List application, which, while representative of many common web development tasks, may not fully encapsulate the challenges of more complex, enterprise-level applications. The findings may not uniformly apply to all types of web projects.
- **Subjectivity in Qualitative Assessments:** Some aspects of the analysis, particularly those related to developer experience and ease of use, inherently involve a degree of subjectivity. While efforts were made to base these assessments on widely acknowledged criteria, individual developer preferences and backgrounds can influence such evaluations.
- **Limited Sample Size in Performance Testing:** The performance benchmarks were conducted on a specific hardware configuration and may not represent the full spectrum of possible deployment environments. A more comprehensive study would involve testing across various devices and network conditions.
- **Ecosystem Volatility:** The analysis of ecosystem and tooling is based on the current state of available libraries and tools. The open-source nature of many of these resources means that their quality, maintenance, and popularity can change rapidly, potentially affecting the ecosystem's overall health.
- **Geographic and Demographic Limitations:** The popularity analysis relies on global surveys and trends, which may not accurately represent regional variations in framework adoption and preferences. Additionally, these surveys may have inherent biases in their respondent demographics.
- **Version-Specific Analysis:** The study focuses on specific versions of React and Angular available at the time of writing. While efforts were made to consider the frameworks' evolutionary trends, future major releases could introduce significant changes not accounted for in this analysis.

- **Limited Cross-Framework Compatibility Analysis:** While the study compares React and Angular, it does not extensively explore scenarios where both frameworks might be used together or in conjunction with other technologies, which could be relevant in some development contexts.
- **Absence of Long-Term Maintenance Perspective:** The analysis primarily focuses on initial development and short-term maintenance. A more comprehensive study would include long-term maintenance considerations over several years, which was beyond the scope of this research.
- **Potential Researcher Bias:** Despite efforts to maintain objectivity, the researcher's prior experience and preferences may inadvertently influence certain aspects of the analysis. This potential bias is acknowledged as a limitation of the study.

These limitations do not invalidate the findings of this comparative study but rather provide context for interpreting the results. They also highlight areas where future research could expand upon this work to provide an even more comprehensive understanding of the relative strengths and weaknesses of React and Angular in various development scenarios.

Readers are encouraged to consider these limitations when applying the insights from this analysis to their specific use cases or decision-making processes. Future studies in this area could address these limitations by expanding the scope, increasing the sample size for performance testing, and conducting longitudinal studies to capture long-term trends in framework evolution and maintenance.

6 DISCUSSION

This comparative study of React and Angular has provided comprehensive insights into the strengths, limitations, and use cases of these two popular front-end frameworks. The analysis encompassed various crucial aspects, including learning curve, performance, ecosystem and tooling, community support and documentation, and popularity. This discussion aims to synthesize these findings and explore their implications for developers, project managers, and organizations in the context of modern web development.

6.1 Learning Curve and Developer Experience

The analysis revealed a significant contrast in the learning experiences associated with React and Angular. React's more gradual learning curve, coupled with its focus on JavaScript fundamentals, positions it as a more accessible option for developers transitioning into modern front-end development. This accessibility is particularly advantageous for projects with tight timelines or teams with varied skill levels. However, React's flexibility, while empowering, can lead to decision fatigue, especially in larger projects where architectural consistency is crucial.

Conversely, Angular's steeper learning curve, attributed to its comprehensive framework approach and reliance on TypeScript, presents initial challenges but offers long-term benefits in code consistency and maintainability. This characteristic makes Angular particularly suitable for large-scale, enterprise-level applications where the investment in learning the framework's intricacies can yield substantial returns in the form of standardized development practices and robust application architecture.

The implications of these learning curve differences extend beyond individual developer preferences. Project managers and team leaders must consider the composition of their development team, the project's timeline, and the long-term maintenance requirements when choosing between these frameworks. For projects requiring rapid prototyping or with developers possessing strong JavaScript backgrounds, React might be the more pragmatic choice. In contrast, for organizations embarking on long-term, complex projects with the resources to invest in comprehensive training, Angular's structured approach could prove more beneficial.

6.2 Performance and Scalability

The performance analysis conducted on the Todo application implementations revealed nuanced differences between React and Angular. React demonstrated superior performance in initial load times, runtime efficiency, and memory usage, suggesting its suitability for applications where speed and resource efficiency are paramount. This performance edge could be particularly advantageous for mobile web applications or projects targeting markets with limited network infrastructure.

Angular, while generally showing higher resource utilization, excelled in scenarios involving large datasets, as evidenced by its superior performance in loading todos. This characteristic aligns with Angular's reputation as a robust solution for data-intensive, enterprise-scale applications. The framework's integrated approach to change detection and its optimization for handling complex data flows contribute to its efficacy in such scenarios.

These performance characteristics have significant implications for application scalability. React's efficient update mechanism and lower memory footprint suggest it might scale more gracefully in applications with frequent, small updates to the UI. Angular's performance in handling larger datasets indicates it might be better suited for applications that scale in terms of data complexity and volume.

However, it's crucial to note that performance optimizations in both frameworks can significantly narrow these gaps. The choice between React and Angular should not solely be based on out-of-the-box performance metrics but should also consider the specific optimization capabilities each framework offers and how they align with the project's scalability requirements.

6.3 Ecosystem and Tooling

The analysis of ecosystem and tooling revealed distinct philosophies underpinning React and Angular. React's ecosystem, characterized by its flexibility and community-driven nature, offers developers a wide array of choices for additional libraries and tools. This flexibility can be a double-edged sword: while it allows for tailored solutions, it also requires more decision-making and potential integration challenges.

Angular's more opinionated and integrated ecosystem provides a more standardized development experience. This approach can lead to increased productivity once the initial learning curve is overcome, as developers can rely on a consistent set of tools and practices across different projects. The implications of these ecosystem differences are particularly relevant for project architecture and long-term maintenance. React's flexible ecosystem may require more careful planning and

documentation of architectural decisions to ensure consistency, especially in larger teams. Angular's integrated approach, while potentially limiting in some aspects, can facilitate easier onboarding of new team members and more straightforward maintenance practices.

Furthermore, the choice of ecosystem can impact the project's ability to adapt to new trends and technologies. React's diverse ecosystem might offer more agility in incorporating cutting-edge solutions, while Angular's more controlled environment could provide more stability and predictability in the face of rapid technological changes.

6.4 Community Support and Documentation

Both React and Angular benefit from robust community support and comprehensive documentation, furthermore with different characteristics. React's larger and more diverse community translates to a wider variety of resources, faster problem-solving, and a constant influx of innovative ideas. This vibrant ecosystem can be particularly beneficial for developers tackling unique challenges or seeking cutting-edge solutions.

Angular's community, while smaller, demonstrates a more focused and enterprise-oriented approach. The framework's comprehensive official documentation and structured learning resources provide a clear path for developers, which can be especially valuable in corporate environments where standardized practices are crucial.

These community dynamics have implications beyond mere problem-solving. They influence the frameworks' evolution, the availability of third-party libraries, and even job market trends. The strong community backing for both frameworks suggests their continued relevance in the foreseeable future, providing some assurance to organizations considering long-term investments in either technology.

6.5 Popularity and Industry Adoption

The popularity analysis revealed React's clear dominance in terms of developer preference and industry adoption. This widespread popularity has several implications:

1. **Talent Pool:** The larger number of React developers potentially makes it easier to find and recruit skilled professionals.

2. **Ecosystem Growth:** Higher popularity often correlates with a more rapidly evolving ecosystem, potentially leading to more innovative solutions and tools.
3. **Long-term Viability:** Widespread adoption can be an indicator of long-term support and continued development of the framework.

However, Angular's strong presence in enterprise environments suggests a different kind of stability. Its adoption by large corporations ensures ongoing development and support, making it a reliable choice for long-term, large-scale projects.

The popularity gap between React and Angular also reflects broader trends in web development, such as the move towards more flexible, component-based architectures. However, it's important to note that popularity should not be the sole criterion for framework selection. The specific needs of a project, the existing skillset of the development team, and the long-term goals of the application should all factor into this decision.

6.6 Limitations and Future Research

While this study provides valuable insights, it's important to acknowledge its limitations. The analysis was based on a specific implementation (a Todo application) and may not fully capture the nuances of more complex, real-world applications. Additionally, the rapid pace of development in both frameworks means that some findings may evolve over time.

Future research could expand on this work by:

1. Conducting longitudinal studies to track how these frameworks evolve over time.
2. Implementing more complex, real-world applications to test the frameworks under diverse scenarios.
3. Exploring the frameworks' performance in specific domains (e.g., e-commerce, data visualization).
4. Investigating the impact of these frameworks on development team dynamics and project management practices.

6.7 Conclusion

The choice between React and Angular is not a matter of determining a universally superior framework, but rather of aligning technology choices with specific project requirements, team capabilities, and organizational goals. React's flexibility and gentle learning curve make it an attractive option for a wide range of projects, particularly those requiring rapid development or targeting resource-constrained environments. Angular's comprehensive approach and robust feature set position it as a strong contender for large-scale, data-intensive applications, especially in enterprise settings where standardization and long-term maintainability are key concerns.

Ultimately, the decision should be based on a holistic evaluation of the project's needs, considering factors such as the application's complexity, the team's expertise, performance requirements, and long-term maintenance plans. Both React and Angular have proven their capability to build powerful, scalable web applications, and both continue to evolve to meet the changing demands of modern web development.

As the web development landscape continues to evolve, staying informed about the latest developments in these frameworks and their ecosystems will be crucial for making informed decisions and leveraging their full potential in creating robust, efficient, and user-friendly web applications.

REFERENCES

1. Strazzullo F 2023 Brief History of Front-end Frameworks. Frameworkless Front-End Development. Apress; 2023. 19–31 p.
2. Fleck R 2019. <https://dribbble.com/>. 2019 Search date 06/04/2024. <https://dribbble.com/stories/2019/11/01/the-evolution-of-web-design-1990s>
3. Wium Lie H 2005. <https://www.wiumlie.no/2006/phd>. 2005 Search date 06/04/2024. <https://www.wiumlie.no/2006/phd/#h-34>
4. MacManus R 2020. <https://webdevelopmenthistory.com>. 2020 Search date 06/04/2024. <https://webdevelopmenthistory.com/1995-the-birth-of-javascript/>
5. jQuery Eases JavaScript, AJAX Development. Search date 06/04/2024. <https://www.eweek.com/development/jquery-eases-javascript-ajax-development/>
6. 2006 | Official jQuery Blog. Search date 06/04/2024. <https://blog.jquery.com/2006/>
7. Dynamic Content Loading with jQuery and AJAX | Reintech media. Search date 06/04/2024. <https://reintech.io/blog/dynamic-content-loading-with-jquery-ajax>
8. 1. Introduction to AngularJS | AngularJS. Search date 06/04/2024. https://learning.oreilly.com/library/view/angularjs/9781449355852/ch01.html#_concepts
9. What is SPA (Single page application) in AngularJS ? - GeeksforGeeks. Search date 06/04/2024. <https://www.geeksforgeeks.org/what-is-spa-single-page-application-in-angularjs/>
10. Introduction to ReactJS: A Guide for Beginners. Search date 06/04/2024. <https://www.microverse.org/blog/introduction-to-reactjs-a-guide-for-beginners>

11. React's Component-Based Architecture: A Case Study | AppMaster. Search date 06/04/2024. <https://appmaster.io/blog/react-component-based-architecture>
12. Introduction to Vue.js 3 - DEV Community. Search date 06/04/2024. <https://dev.to/lenil-doluan/introduction-to-vuejs-22i6>
13. Introduction — Vue.js. Search date 06/04/2024. <https://v2.vuejs.org/v2/guide/index.html>
14. What is Vue.js in Frontend Development? - LoadFocus. Search date 06/04/2024. <https://loadfocus.com/glossary/vue-js>
15. Angular (web framework) - Wikipedia. Search date 06/04/2024. [https://en.wikipedia.org/wiki/Angular_\(web_framework\)](https://en.wikipedia.org/wiki/Angular_(web_framework))
16. TypeScript: Documentation - Angular. Search date 06/04/2024. <https://www.typescript-lang.org/docs/handbook/angular.html>
17. The History of React.js on a Timeline - RisingStack Engineering. Search date 06/04/2024. <https://blog.risingstack.com/the-history-of-react-js-on-a-timeline/>
18. Introducing Hooks – React. Search date 06/04/2024. <https://legacy.reactjs.org/docs/hooks-intro.html>
19. Reactivity as the catalyst: Driving Evolution in Frameworks like React, Vue, Svelte, and Angular | by DadCod | Apr, 2024 | Medium. Search date 06/04/2024. <https://medium.com/@dadcod/reactivity-as-the-catalyst-driving-evolution-in-frameworks-like-react-vue-svelte-and-angular-002f2a54a895>
20. Unidirectional and Bidirectional Data Flow - The Ultimate Front-End Interview Questions Guide Pt: 1_reactnative_weixin_0010034-React. Search date 06/04/2024. <https://devpress.csdn.net/react/62eb67ef20df032da732b264.html>
21. <https://infinum.com> [Internet] 2022. 2022 Search date 06/04/2024. <https://infinum.com/handbook/frontend/react/react-hooks>

22. Hossain S 2021. <https://saikat-hossain.medium.com>. 2021 Search date 06/04/2024. <https://saikat-hossain.medium.com/react-lifecycle-bf83b9832be8>
23. Strumpflohner J 2016. juri.dev. 2016 Search date 06/04/2024. <https://juri.dev/blog/2016/09/ng2-released/>
24. AngularJS - Wikipedia. Search date 06/04/2024. <https://en.wikipedia.org/wiki/AngularJS>
25. Pietrucha B 2019. <https://dev-academy.com>. 2019 Search date 06/04/2024. <https://dev-academy.com/angular-architecture-best-practices/>
26. Ankush A 2021. <https://medium.com>. 2021 Search date 06/04/2024. <https://medium.com/front-end-weekly/smart-dumb-presentational-components-in-angular-bf01c5bf2d40>
27. Angular Template - javatpoint. Search date 06/04/2024. <https://www.javatpoint.com/angular-template>
28. dhananjay kumar. <https://www.infragistics.com>. Search date 06/04/2024. https://www.infragistics.com/community/blogs/b/dhananjay_kumar/posts/introduction-to-angular-2-components
29. Dhaduk H 2020. <https://hashnode.com/>. 2020 Search date 06/04/2024. <https://hashnode.com/post/data-binding-and-server-rendering-in-angularjs-and-reactjs-ck6btivfk001r3ns1g2vw33sg>
30. Angular Team Architecture Overview - ts - GUIDE. Search date 06/04/2024. <https://v2.angular.io/docs/ts/latest/guide/architecture.html>
31. Angular Team 2022. <https://v17.angular.io>. 2022 Search date 06/04/2024. <https://v17.angular.io/tutorial/tour-of-heroes>

32. shobhnat 2018. <https://www.loginworks.com>. 2018 Search date 06/04/2024. <https://www.loginworks.com/blogs/how-to-develop-a-single-page-web-application-using-angular-6-0/>
33. JavaScript Programming - The State of Developer Ecosystem in 2023 Infographic | JetBrains: Developer Tools for Professionals and Teams. Search date 08/04/2024. https://www.jetbrains.com/lp/devecosystem-2023/javascript/#js_frameworks
34. Stack Overflow Developer Survey 2023. Search date 08/04/2024. <https://survey.stackoverflow.co/2023/#most-popular-technologies-webframe>
35. State of JavaScript 2023: Front-end Frameworks. Search date 08/04/2024. <https://2023.stateofjs.com/en-US/libraries/front-end-frameworks/>
36. React, Angular - Explore - Google Trends. Search date 08/04/2024. <https://trends.google.com/trends/explore?cat=5&date=today%205-y&q=React,Angular#TIMESERIES>
37. JavaScript technologies Web Usage Distribution. Search date 08/05/2024. <https://trends.builtwith.com/javascript>
38. Websites using React. Search date 08/05/2024. <https://trends.builtwith.com/websitelist/React>

APPENDICES

React todo application code appendix 1

Angular todo application code appendix 2

State of javascript 2023 front-end frameworks ratios over time json data appendix 3

Detailed performance benchmark data appendix 4

```
//app.jsx

import './App.css';

import { useEffect, useState } from 'react';
import { useCookies } from 'react-cookie';
import ListHeader from './components/ListHeader';
import ListItem from './components/ListItem';
import Auth from './components/Auth';

const App = () => {
  const [tasks, setTasks] = useState(null);
  const [cookies, setCookie, removeCookie] = useCookies('user');

  const authToken = cookies.AuthToken;
  const userEmail = cookies.Email;

  const getData = async () => {
    try {
      const response = await fetch(
        `${import.meta.env.VITE_APP_SERVERURL}/todos/${userEmail}`
      );
      const json = await response.json();

      setTasks(json);
    } catch (err) {
      console.error(err);
    }
  };

  useEffect(() => {
    if (authToken) {
      getData();
    }
  }, []);

  console.log(tasks);

  const sortedTasks = tasks?.sort(
    (a, b) => new Date(a.date) - new Date(b.date)
  );
}
```

```

    );

    return (
      <div>
        <div>
          <h1 className="main-title">Todo App </h1>
        </div>
        <div className="app">
          <div className="todo-main-container">
            {!authToken && <Auth />}
            {authToken && (
              <>
                <ListHeader listName={`My Todo Tasks`} getData={getData} />
                <p className="user-email">Welcome back {userEmail}</p>
                {sortedTasks?.map((task) => (
                  <ListItem key={task.id} task={task} getData={getData} />
                ))}
              </>
            )}
            <p className="copyright">© 2024 Sanjay Khatri Chhetri </p>
          </div>
        </div>
      </div>
    );
  };
};

export default App;

//main.jsx
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

```

```

    </React.StrictMode>,
  )

//src/components/ListItem.jsx
import { useState } from "react";
import React from "react";
import TickIcon from "../TickIcon";
import ProgressBar from "../ProgressBar";
import Modal from "../Modal";

const ListItem = ({ task, getData }) => {
  const [showModal, setShowModal] = useState(false);

  const deleteItem = async () => {
    try {
      const response = await fetch(
        `${import.meta.env.VITE_APP_SERVERURL}/todos/${task.id}`,
        {
          method: "DELETE",
        }
      );
      if (response.status === 200) {
        console.log("I can now delete todos from UI");
        getData();
      }
    } catch (err) {
      console.error(err);
    }
  };

  return (
    <li className="list-item">
      <div className="info-container">
        <TickIcon />
        <p className="task-title">{task.title}</p>
        <ProgressBar progress={task.progress}

```

```

    />
  </div>

  <div className="button-container">
    <button className="edit" onClick={() => setShowModal(true)}>
      Edit
    </button>
    <button className="delete" onClick={deleteItem}>
      Delete
    </button>
  </div>

  {showModal && (
    <Modal
      mode="edit"
      setShowModal={setShowModal}
      getData={getData}
      task={task}
    />
  )}
</li>
);
};

export default ListItem;

//src/components/ListHeader.jsx
import React from "react";
import Modal from "../Modal";
import { useState } from "react";
import { useCookies } from "react-cookie";

const ListHeader = ({ listName, getData }) => {
  const [showModal, setShowModal] = useState(false);
  const [cookies, setCookie, removeCookies] = useCookies(null);

  const signOut = () => {
    removeCookies("Email");
  }

```

Full source Code can be found in [GitHub for my React frontend and backend todo app found at same repository.](#)


```
//App.config.ts

import {
  ApplicationConfig,
  provideZoneChangeDetection,
  importProvidersFrom,
  isDevMode,
} from '@angular/core';
import { provideRouter } from '@angular/router';
import {
  provideHttpClient,
  withFetch,
  withInterceptors,
} from '@angular/common/http';
import { provideClientHydration } from '@angular/platform-browser';
import { routes } from './app.routes';
import { authInterceptor } from './interceptors/auth.interceptor';
import { provideAnimationsAsync } from '@angular/platform-browser/animations/async';
import {
  MatDialogModule,
  MAT_DIALOG_DEFAULT_OPTIONS,
} from '@angular/material/dialog';
import { MatButtonModule } from '@angular/material/button';
import { MatInputModule } from '@angular/material/input';
import { MatFormFieldModule } from '@angular/material/form-field';
import { MatSelectModule } from '@angular/material/select';
import { provideServiceWorker } from '@angular/service-worker';

export const appConfig: ApplicationConfig = {
  providers: [
    provideServiceWorker('ngsw-worker.js', {
      enabled: !isDevMode(),
      registrationStrategy: 'registerWhenStable:30000',
    }),
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideClientHydration(),
    provideHttpClient(withFetch(), withInterceptors([authInterceptor])),
    provideAnimationsAsync(),
    importProvidersFrom(
      MatDialogModule,
      MatButtonModule,
```

```
// src/app/services/auth.service.ts

import { Injectable, PLATFORM_ID, Inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { BehaviorSubject, Observable } from 'rxjs';
import { tap } from 'rxjs/operators';
import { Router } from '@angular/router';
import { isPlatformBrowser } from '@angular/common';
import { environment } from '../../../environments/environment';

interface AuthResponse {
  token: string;
  user: any; // Replace 'any' with a proper user interface if you have one
}

@Injectable({
  providedIn: 'root',
})
export class AuthService {
  private apiUrl = environment.apiUrl;
  private currentUserSubject: BehaviorSubject<any>;
  public currentUser: Observable<any>;
  private isBrowser: boolean;

  constructor(
    private http: HttpClient,
    private router: Router,
    @Inject(PLATFORM_ID) platformId: Object
  ) {
    this.isBrowser = isPlatformBrowser(platformId);
    this.currentUserSubject = new BehaviorSubject<any>(
      this.getUserFromStorage()
    );
    this.currentUser = this.currentUserSubject.asObservable();
  }

  private getUserFromStorage(): any {
    if (this.isBrowser) {
      const storedUser = localStorage.getItem('currentUser');
      return storedUser ? JSON.parse(storedUser) : null;
    }
    return null;
  }
}
```

```

//app.component.ts

import { Component } from '@angular/core';
import { RouterOutlet, RouterLink } from '@angular/router';
import { NavComponent } from '../components/nav/nav.component';
import { OfflineIndicatorComponent } from '../offline-indicator/offline-
indicator.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, NavComponent, OfflineIndicatorComponent],
  template: `
    <app-nav></app-nav>
    <router-outlet></router-outlet>
  `,
  styles: [],
})
export class AppComponent {
  title = 'todo-frontend-angular';
}

//src/app/app.component.ts
import { Component } from '@angular/core';

import { RouterOutlet, RouterLink } from '@angular/router';
import { NavComponent } from '../components/nav/nav.component';
import { OfflineIndicatorComponent } from '../offline-indicator/offline-
indicator.component';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet, NavComponent, OfflineIndicatorComponent],
  template: `
    <app-nav></app-nav>
    <router-outlet></router-outlet>
    <app-offline-indicator></app-offline-indicator>
  `,
  styles: [],
})
export class AppComponent {
  title = 'todo-frontend-angular';
}

```

Source code for angular todo app [frontend](#) and [backend](#) can be found in GitHub and are separate.

```
[
  {
    "dataPath":
"dataAPI.surveys.state_of_js.js2023.front_end_frameworks.front_end_frameworks_ratios",
    "name": "front_end_frameworks_ratios",
    "data": [
      {
        "id": "react",
        "entity": {
          "name": "React",
          "nameClean": "React",
          "entityType": "library",
          "description": "A declarative, efficient, and flexible JavaScript library for
building user interfaces.",
          "descriptionClean": "A declarative, efficient, and flexible JavaScript library
for building user interfaces.",
          "homepage": {
            "url": "https://reactjs.org/"
          }
        },
        "responses": {
          "allEditions": [
            {
              "year": 2016,
              "editionId": "js2016",
              "ratios": {
                "awareness": 0.99,
                "interest": 0.75,
                "usage": 0.52,
                "retention": 0.92,
                "positivity": 0.83
              },
              "completion": {
                "count": 10257,
                "percentageSurvey": 99.99,
                "total": 10258
              }
            },
            {
              "year": 2017,
              "editionId": "js2017",
              "ratios": {
                "awareness": 0.99,
```

This appendix contains the raw data collected during the performance comparison of React and Angular frameworks. The data was collected using Chrome DevTools and Lighthouse on a system with the following specifications:

- Operating System: Microsoft Windows 11 Pro
- Processor: Intel(R) Core (TM) i5-9300H CPU @ 2.40GHz, 2400 MHz, 4 Core(s), 8 Logical Processor(s)
- Installed Physical Memory (RAM): 16.0 GB
- Storage: Samsung SSD 980 500GB m.2
- Browser: Microsoft Edge for Business Version 127.0.2651.86 (Official build) (64-bit)

3.1 Lighthouse Performance Tests

Table 6. React Raw Lighthouse Performance Metrics

Metric	Test 1	Test 2	Test 3	Test 4	Test 5
First Contentful Paint	1.0 s	0.9 s	0.9 s	1.0 s	1.0 s
Largest Contentful Paint	1.5 s	1.5 s	1.5 s	1.5 s	1.5 s
Total Blocking Time	0ms	10ms	0ms	20ms	20ms
Cumulative Layout Shift	0.005	0	0.005	0.005	0.005
Speed Index	1.0 s	1.0 s	0.9s	1.1 s	1.0 s

Table 7. Angular Raw Lighthouse Performance Metrics

Metric	Test 1	Test 2	Test 3	Test 4	Test 5
First Contentful Paint	1.4 s	0.9 s	1.8 s	0.9 s	0.9 s
Largest Contentful Paint	3.9s	3.5 s	3.5 s	3.5 s	3.5 s
Total Blocking Time	60ms	40ms	120msms	70ms	100ms
Cumulative Layout Shift	0	0	0	0	0
Speed Index	1.4 s	1.0 s	1.8 s	0,9 s	1.1s

3.2 Operation-Specific Performance Data

Table 8. React Raw Operation Performance (in milliseconds)

Operation	Test 1	Test 2	Test 3	Test 4	Test 5
Loading Todos	14321	14511	13224	13250	13230
Creating Todo	10912	8739	10291	10952	18371
Editing Todo	14870	12211	13245	14671	30756
Deleting Todo	3634	3634	3634	3634	3821

Table 9. React Performance Data for **Loading Todos** (in milliseconds)

Metric	Loading	Scripting	Rendering	Painting	System	Idle	Total
1	14	321	45	1	178	3284	3844
2	10	251	45	1	157	1044	1509
3	16	224	39	1	171	1091	1543
4	13	250	44	1	170	1046	1523
5	12	254	50	2	179	1091	1588
Average	13	260	44.6	1.2	171	1511	2001

Table 10. React Performance Data for **Creating Todo** (in milliseconds)

Metric	Loading	Scripting	Rendering	Painting	System	Idle	Total
1	3	368	54	47	309	10132	10912
2	2	346	41	26	274	8050	8739
3	2	338	44	36	277	9594	10291
4	2	421	52	44	325	10107	10952
5	3	521	54	65	348	17380	18371
Average	2.4	398.8	49	43.6	306.6	11052.6	11853

Table 11.React Performance Data for **Editing Todo** (in milliseconds)

Metric	Loading	Scripting	Rendering	Painting	System	Idle	Total
1	3	462	62	71	359	13913	14870
2	3	411	61	68	340	11326	12211
3	2	453	66	70	389	12265	13245
4	3	425	62	68	338	13776	14671
5	3	587	106	153	500	29407	30756
Average	2.8	467.6	71.4	86	385.2	16137.4	17150.6

Table 12.React Performance Data for **Deleting Todo** (in milliseconds)

Iteration	Loading	Scripting	Rendering	Painting	System	Idle	Total
1	3	292	15	12	234	3078	3634
2	4	285	13	11	246	2813	3373
3	2	279	15	13	288	3047	3644
4	2	250	10	7	234	2846	3349
5	4	311	18	12	276	3200	3821
Average	3	283.4	14.2	11	255.6	2996.8	3564.2

Analysis:

Table 13. Angular Raw Operation Performance (in milliseconds)

Operation	Test 1	Test 2	Test 3	Test 4	Test 5
Loading Todos	885	883	873	952	923
Creating Todo	33575	20837	23674	29900	31708
Editing Todo	20252	23253	17942	22556	27648
Deleting Todo	5615	3513	3745	3119	4174

Table 14.Angular performance data for **Loading Todos** (in milliseconds)

Metric	Test 1	Test 2	Test 3	Test 4	Test 5	Average
Loading	18 ms	22 ms	21 ms	22 ms	21 ms	20.8 ms

Scripting	319 ms	343 ms	320 ms	366 ms	353 ms	340.2 ms
Rendering	57 ms	58 ms	51 ms	57 ms	55 ms	55.6 ms
Painting	12 ms	13 ms	11 ms	13 ms	11 ms	12 ms
System	292 ms	291 ms	295 ms	302 ms	309 ms	297.8 ms
Idle	157 ms	156 ms	174 ms	192 ms	174 ms	170.6 ms
Total	855 ms	883 ms	873 ms	952 ms	923 ms	897.2 ms

Table 15.Angular Performance data for **Creating Todo** (in milliseconds)

Metric	Test 1	Test 2	Test 3	Test 4	Test 5	Average
Loading	4 ms	6 ms	5 ms	5 ms	6 ms	5.2 ms
Scripting	911 ms	757 ms	760 ms	775 ms	922 ms	825 ms
Rendering	310 ms	255 ms	235 ms	246 ms	297 ms	268.6 ms
Painting	208 ms	163 ms	157 ms	132 ms	185 ms	169 ms
System	1188 ms	938 ms	972 ms	1042 ms	1103 ms	1048.6 ms
Idle	30954 ms	18718 ms	21545 ms	27699 ms	29196 ms	25622.4 ms
Total	33575 ms	20837 ms	23674 ms	29900 ms	31708 ms	27938.8 ms

Table 16.Angular performance data for **Editing Todo** (in millisecond)

Metric	Test 1	Test 2	Test 3	Test 4	Test 5	Average
Loading	0 ms	0 ms	7 ms	0 ms	5 ms	2.4 ms
Scripting	252 ms	314 ms	855 ms	455 ms	739 ms	523 ms
Rendering	376 ms	368 ms	331 ms	312 ms	294 ms	336.2 ms
Painting	249 ms	221 ms	232 ms	188 ms	182 ms	214.4 ms
System	661 ms	712 ms	1081 ms	642 ms	1189 ms	857 ms
Idle	18713 ms	21638 ms	15437 ms	20960 ms	25238 ms	20397.2 ms
Total	20252 ms	23253 ms	17942 ms	22556 ms	27648 ms	22330.2 ms

Table 17. Angular performance data for **Deleting Todo** (in millisecond)

Metric	Test 1	Test 2	Test 3	Test 4	Test 5	Average
Loading	5 ms	6 ms	3 ms	5 ms	6 ms	5 ms
Scripting	364 ms	321 ms	316 ms	335 ms	340 ms	335.2 ms
Rendering	99 ms	60 ms	55 ms	55 ms	68 ms	67.4 ms
Painting	19 ms	10 ms	11 ms	10 ms	14 ms	12.8 ms
System	648 ms	552 ms	516 ms	604 ms	614 ms	586.8 ms
Idle	4480 ms	2563 ms	2842 ms	2109 ms	3132 ms	3025.2 ms
Total	5615 ms	3513 ms	3745 ms	3119 ms	4174 ms	4033.2 ms